Mastering Java Streams & Functional Programming -- Complete Professional Guide

---

Author: Senior Java Backend Architect **Target Audience:** Java developers with 1--5 years of experience aiming for mastery in functional programming and Java Streams **Prerequisites:** Core Java, OOP fundamentals, Java Collections basics **Java Version:** Java 8+ (with modern best practices up to Java 17+)

Table of Contents

1. Introduction to Functional Programming in Java
2. Java Stream API Overview
3. Stream Creation Methods
4. All Intermediate Stream Operations
5. All Terminal Stream Operations
6. Collectors Deep Dive
7. Functional Interfaces (Complete Guide)
8. Method References
9. Comparable vs Comparator (Deep Guide)
10. Advanced Stream Topics
11. Real-World Use Cases
12. 50+ Coding Interview Problems Using Streams
13. Stream Best Practices
14. Common Interview Questions
15. Practice Section

Part 1: Introduction to Functional Programming in Java

1.1 What is Functional Programming?

Concept Explanation

Functional Programming (FP) is a programming paradigm that treats computation as the evaluation of mathematical functions. Instead of changing state and mutating data, FP emphasizes **pure functions**, **immutability**, and **declarative expressions**.

Core Principles of Functional Programming

Principle	Description	Java Example
Pure Functions	Same input always produces same output, no side effects	`x -> x * 2`
Immutability	Data does not change after creation	`List.of(1, 2, 3)` (unmodifiable)
First-Class Functions	Functions can be passed as arguments and returned	`Function<Integer, Integer> fn = x -> x + 1;`
Higher-Order Functions	Functions that take or return other functions	`list.stream().map(fn)`
Declarative Style	Describe WHAT to do, not HOW	`stream.filter().map().collect()`
No Side Effects	Functions don't modify external state	No global variable mutation

1.2 Why Java Introduced Functional Programming

The Problem Before Java 8

```
Before Java 8:
  X Verbose anonymous inner classes for simple operations
  X No way to pass behavior (functions) as parameters easily
  X Imperative loops for collection processing
  X No built-in parallel processing for collections
  X Boilerplate code for simple data transformations
```

```
After Java 8:
  [ok] Lambda expressions for concise function definitions
  [ok] Functional interfaces (Predicate, Function, Consumer, Supplier)
  [ok] Stream API for declarative data processing
  [ok] Method references for even cleaner code
  [ok] Built-in parallel stream support
```

Before vs After Java 8

```java
// X BEFORE Java 8: Verbose, imperative
List<String> names = new ArrayList<>();
for (Employee emp : employees) {
    if (emp.getSalary() > 50000) {
        names.add(emp.getName().toUpperCase());
    }
}
Collections.sort(names);

// [ok] AFTER Java 8: Concise, declarative
List<String> names = employees.stream()
    .filter(emp -> emp.getSalary() > 50000)
    .map(emp -> emp.getName().toUpperCase())
    .sorted()
    .collect(Collectors.toList());
```

1.3 Imperative vs Declarative Programming

Side-by-Side Comparison

Aspect	Imperative	Declarative (Functional)
Focus	HOW to do it	WHAT to do
State	Mutable variables	Immutable transformations
Control	Loops (for, while)	Stream operations (map, filter)
Readability	Step-by-step instructions	Pipeline of transformations
Parallelism	Manual thread management	`parallelStream()` built-in
Side Effects	Common and expected	Minimized or eliminated

Example: Find the sum of squares of even numbers

```
// IMPERATIVE: Tell the computer HOW
int sum = 0;
for (int n : numbers) {
    if (n % 2 == 0) {
        sum += n * n;
    }
}

// DECLARATIVE: Tell the computer WHAT
int sum = numbers.stream()
    .filter(n -> n % 2 == 0)
    .mapToInt(n -> n * n)
    .sum();
```

1.4 Key Concepts in Functional Programming

Lambda Expressions

A **lambda expression** is a concise way to represent an anonymous function.

```
// Syntax: (parameters) -> expression
// Or:     (parameters) -> { statements; }

// No parameters
Runnable task = () -> System.out.println("Running");

// One parameter (parentheses optional)
Function<String, Integer> length = s -> s.length();

// Multiple parameters
BiFunction<Integer, Integer, Integer> add = (a, b) -> a + b;

// Multiple statements
Comparator<String> comp = (s1, s2) -> {
    int lenDiff = s1.length() - s2.length();
    return lenDiff != 0 ? lenDiff : s1.compareTo(s2);
};
```

Effectively Final Variables

Lambda expressions can access local variables only if they are **effectively final** (not modified after initialization).

```
int threshold = 50000;  // Effectively final -- never reassigned

// [ok] Works: threshold is effectively final
employees.stream()
    .filter(e -> e.getSalary() > threshold)
    .collect(Collectors.toList());

// X Compilation error: threshold is modified
// threshold = 60000;
// employees.stream().filter(e -> e.getSalary() > threshold);
```

1.5 Benefits in Real Systems

Benefit	How It Helps
Readability	Pipelines read like English: "filter employees, map to names, collect to list"
Conciseness	10 lines of imperative code -> 3 lines with streams
Parallelism	Switch `stream()` to `parallelStream()` for free parallelism
Composability	Chain operations easily: filter -> map -> sort -> collect
Testability	Pure functions are trivially testable
Maintainability	Declarative code is easier to understand and modify

1.6 Real-World Use Cases

```
1. DATA PROCESSING PIPELINES
   API Response -> Filter invalid -> Transform -> Group -> Return
   Example: Process 10,000 API records, filter nulls, transform to DTOs

2. MICROSERVICE DATA TRANSFORMATION
   Service A Response -> Map to Internal DTO -> Validate -> Persist
   Example: Transform external payment gateway response to internal model
```

```
3. REPORTING & ANALYTICS
   Database Results -> Group by category -> Aggregate -> Sort -> Export
   Example: Monthly sales report grouped by region

4. LOG PROCESSING
   Log entries -> Filter by severity -> Extract patterns -> Count occurrences
   Example: Analyze error logs to find most frequent exceptions

5. CONFIGURATION PROCESSING
   Raw configs -> Validate -> Filter active -> Convert to runtime objects
   Example: Process feature flags from config server
```

Part 2: Java Stream API Overview

2.1 What are Streams?

Concept Explanation

A **Stream** is a sequence of elements that supports functional-style aggregate operations. It is NOT a data structure -- it does not store data. Instead, it provides a declarative pipeline for processing data from a source (like a collection, array, or I/O channel).

Streams vs Collections

Aspect	Collection	Stream
Purpose	Store and manage data	Process and compute data
Storage	Holds elements in memory	Does NOT store elements
Consumption	Can be iterated multiple times	Can be consumed ONLY ONCE
Modification	Elements can be added/removed	Does NOT modify the source
Evaluation	Eager (immediate)	Lazy (deferred until terminal op)
Size	Finite	Can be infinite
Iteration	External (you control loop)	Internal (stream controls iteration)

Key Characteristics

```
Stream Characteristics:
  1. NO STORAGE        -> Streams don't store data, they pipeline it
  2. FUNCTIONAL        -> Operations produce results without modifying source
  3. LAZY              -> Intermediate operations are not executed until
terminal op
  4. POSSIBLY UNBOUNDED -> Streams can be infinite (Stream.generate,
Stream.iterate)
  5. CONSUMABLE        -> A stream can only be traversed once; create a new one
to re-traverse
  6. PIPELINE          -> Operations are chained into a pipeline: source ->
intermediate -> terminal
```

2.2 Why Streams are Used

```
Problem: Process a list of 100,000 employees

Without Streams (Imperative):
  Step 1: Loop through all employees
  Step 2: Check salary > 50000 (if statement)
  Step 3: Create name string (string operation)
  Step 4: Add to temporary list
  Step 5: Sort the temporary list
  Step 6: Take first 10
  -> 15+ lines of code, 3 temporary variables, hard to parallelize

With Streams (Declarative):
  employees.stream()
      .filter(e -> e.getSalary() > 50000)
      .map(Employee::getName)
      .sorted()
      .limit(10)
      .collect(Collectors.toList());
  -> 6 lines, no temporary variables, trivially parallelizable
```

2.3 Internal vs External Iteration

```
EXTERNAL ITERATION (Traditional):
  You (the programmer) control the iteration:

  for (Employee emp : employees) {     // YOU iterate
      if (emp.getSalary() > 50000) {   // YOU filter
          names.add(emp.getName());     // YOU collect
      }
  }

INTERNAL ITERATION (Streams):
  The Stream API controls the iteration:

  employees.stream()                   // Stream iterates internally
      .filter(e -> e.getSalary() > 50000)  // Stream filters
      .map(Employee::getName)               // Stream maps
      .collect(Collectors.toList());        // Stream collects

  Why Internal Iteration is Better:
```

```
  [ok] Stream can optimize iteration order
  [ok] Stream can parallelize automatically
  [ok] Stream can short-circuit (stop early when result is found)
  [ok] Stream can fuse operations (filter + map in single pass)
```

2.4 Lazy Evaluation

How Lazy Evaluation Works

```
Given this pipeline:
  List<String> result = names.stream()     // Source: ["Alice", "Bob",
"Charlie", "David"]
      .filter(n -> n.length() > 3)          // Intermediate (LAZY)
      .map(String::toUpperCase)             // Intermediate (LAZY)
      .collect(Collectors.toList());         // Terminal (TRIGGERS execution)

Execution with Lazy Evaluation:
  "Alice"   -> filter(length > 3) -> YES -> map(toUpperCase) -> "ALICE" -> add
to result
  "Bob"     -> filter(length > 3) -> NO  -> SKIP (map never called)
  "Charlie" -> filter(length > 3) -> YES -> map(toUpperCase) -> "CHARLIE" -> add
to result
  "David"   -> filter(length > 3) -> YES -> map(toUpperCase) -> "DAVID" -> add
to result

Key Insight:
  [ok] "Bob" was filtered out -- map() was NEVER called for "Bob"
  [ok] Each element is fully processed through the pipeline before the next
element starts
  [ok] Without terminal operation, NOTHING executes
  [ok] This is called "loop fusion" -- multiple operations are combined into a
single pass
```

Demonstrating Laziness

```java
// This does NOTHING -- no terminal operation triggers execution
Stream<String> stream = names.stream()
    .filter(n -> {
        System.out.println("Filtering: " + n);  // Never printed!
        return n.length() > 3;
    })
    .map(n -> {
```

```java
        System.out.println("Mapping: " + n);    // Never printed!
        return n.toUpperCase();
    });
// No output! Stream operations are lazy.

// NOW add terminal operation -- execution is triggered
List<String> result = stream.collect(Collectors.toList());
// Output:
//   Filtering: Alice
//   Mapping: Alice
//   Filtering: Bob         filtered out, map not called
//   Filtering: Charlie
//   Mapping: Charlie
//   Filtering: David
//   Mapping: David
```

2.5 Stream Pipeline Architecture

Pipeline Structure

```
+------------------+      +-----------------------+      +------------------+
|      SOURCE      | ---> |   INTERMEDIATE OPS    | ---> |   TERMINAL OP    |
|                  |      |   (0 or more, LAZY)   |      |   (exactly 1)    |
+------------------+      +-----------------------+      +------------------+
|                  |      |                       |      |                  |
| Collection       |      | filter()              |      | collect()        |
| Array            |      | map()                 |      | forEach()        |
| I/O Channel      |      | flatMap()             |      | reduce()         |
| Generator        |      | sorted()              |      | count()          |
|                  |      | distinct()            |      | min() / max()    |
|                  |      | peek()                |      | findFirst()      |
|                  |      | limit() / skip()      |      | anyMatch()       |
+------------------+      +-----------------------+      +------------------+

RULES:
  1. A pipeline must have exactly ONE source
  2. A pipeline may have ZERO or MORE intermediate operations
  3. A pipeline must have exactly ONE terminal operation
  4. Intermediate operations return a new Stream (allowing chaining)
  5. Terminal operations produce a result or side effect (NOT a Stream)
  6. After a terminal operation, the stream is CONSUMED and cannot be reused
```

Stream Reuse Error

```java
Stream<String> stream = names.stream().filter(n -> n.length() > 3);

// First terminal operation -- OK
List<String> result1 = stream.collect(Collectors.toList());

// Second terminal operation -- IllegalStateException!
// long count = stream.count();  // X Stream already consumed!

// Solution: Create a new stream
long count = names.stream().filter(n -> n.length() > 3).count();
```

Part 3: Stream Creation Methods

3.1 Collection.stream()

Syntax & Explanation

Creates a sequential stream from any `Collection` (List, Set, Queue, etc.).

```
List<String> names = Arrays.asList("Alice", "Bob", "Charlie");
Stream<String> stream = names.stream();
```

Real-World Use Case

```
// Processing a list of orders from database
List<Order> orders = orderRepository.findAll();

List<OrderDTO> activeOrders = orders.stream()
    .filter(order -> order.getStatus() == OrderStatus.ACTIVE)
    .map(order -> new OrderDTO(order.getId(), order.getAmount(),
order.getCustomerName()))
    .sorted(Comparator.comparing(OrderDTO::getAmount).reversed())
    .collect(Collectors.toList());
```

> ***Interview Tip:*** `stream()` *is the most commonly used stream creation method. It creates a sequential stream, meaning elements are processed one at a time in encounter order.*

3.2 Collection.parallelStream()

Syntax & Explanation

Creates a parallel stream that can process elements concurrently using the ForkJoinPool.

```
List<String> names = Arrays.asList("Alice", "Bob", "Charlie");
```

```
Stream<String> parallelStream = names.parallelStream();
```

Real-World Use Case

```
// Processing large dataset -- CPU-intensive transformation
List<Report> monthlyReports = rawDataList.parallelStream()
    .filter(data -> data.getYear() == 2024)
    .map(data -> generateReport(data))    // CPU-intensive operation
    .collect(Collectors.toList());
```

> ***Interview Discussion:*** `parallelStream()` *uses a shared* `ForkJoinPool.commonPool()` *by default. Be cautious: it can actually be SLOWER for small datasets or I/O-bound operations due to thread management overhead. The threshold is typically around 10,000+ elements with CPU-intensive operations.*

3.3 Stream.of()

Syntax & Explanation

Creates a stream from explicitly listed elements.

```
// Single element
Stream<String> single = Stream.of("Hello");

// Multiple elements
Stream<String> multiple = Stream.of("Alice", "Bob", "Charlie");

// From array
String[] arr = {"A", "B", "C"};
Stream<String> fromArray = Stream.of(arr);
```

Real-World Use Case

```
// Combining multiple configuration sources
Stream<String> configSources = Stream.of(
    "application.yml",
```

```
    "application-dev.yml",
    "application-local.yml"
);

List<Properties> configs = configSources
    .map(this::loadConfig)
    .filter(Objects::nonNull)
    .collect(Collectors.toList());
```

> ***Interview Tip:*** `Stream.of()` *is useful for creating small, known streams. For empty streams, use* `Stream.empty()`*. For a single element,* `Stream.of(element)` *is preferred over creating a list.*

3.4 Arrays.stream()

Syntax & Explanation

Creates a stream from an array. Supports both object arrays and primitive arrays.

```
// Object array
String[] names = {"Alice", "Bob", "Charlie"};
Stream<String> nameStream = Arrays.stream(names);

// Primitive array -- returns IntStream, LongStream, or DoubleStream
int[] numbers = {1, 2, 3, 4, 5};
IntStream intStream = Arrays.stream(numbers);

// Partial array (start inclusive, end exclusive)
IntStream partial = Arrays.stream(numbers, 1, 4);  // [2, 3, 4]
```

Real-World Use Case

```
// Processing CSV row data
String[] csvRow = line.split(",");
Map<String, String> rowMap = IntStream.range(0, headers.length)
    .boxed()
    .collect(Collectors.toMap(
        i -> headers[i].trim(),
```

```
        i -> i < csvRow.length ? csvRow[i].trim() : ""
    ));
```

> ***Interview Discussion:*** *For primitive arrays,* `Arrays.stream()` *returns specialized streams (* `IntStream` *,* `LongStream` *,* `DoubleStream` *) which avoid autoboxing overhead and provide additional methods like* `sum()` *,* `average()` *.*

3.5 Files.lines()

Syntax & Explanation

Creates a stream of lines from a file. Each line becomes an element in the stream. The stream is lazy -- lines are read on demand.

```
// Basic usage (must handle IOException)
try (Stream<String> lines = Files.lines(Paths.get("data.txt"))) {
    lines.filter(line -> !line.isEmpty())
         .map(String::trim)
         .forEach(System.out::println);
}

// With charset
try (Stream<String> lines = Files.lines(Paths.get("data.txt"),
StandardCharsets.UTF_8)) {
    long count = lines.count();
}
```

Real-World Use Case

```
// Analyzing log files for error patterns
try (Stream<String> logLines = Files.lines(Paths.get("/var/log/app.log"))) {
    Map<String, Long> errorCounts = logLines
        .filter(line -> line.contains("ERROR"))
        .map(line -> line.substring(line.indexOf("ERROR") + 6).split(":")
[0].trim())
        .collect(Collectors.groupingBy(
            Function.identity(),
            Collectors.counting()
        ));
```

```
    errorCounts.entrySet().stream()
        .sorted(Map.Entry.<String, Long>comparingByValue().reversed())
        .limit(10)
        .forEach(e -> System.out.println(e.getKey() + ": " + e.getValue()));
}
```

> ***Interview Tip:*** *Always use try-with-resources with* `Files.lines()` *because it opens a file handle. Forgetting to close can cause resource leaks. The stream is lazy, so the entire file is NOT loaded into memory.*

3.6 Stream.generate()

Syntax & Explanation

Creates an infinite stream by repeatedly invoking a `Supplier`. Must be limited with `limit()` to avoid infinite processing.

```
// Generate infinite stream of random numbers
Stream<Double> randoms = Stream.generate(Math::random);

// Always use limit() with generate()
List<Double> fiveRandoms = Stream.generate(Math::random)
    .limit(5)
    .collect(Collectors.toList());

// Generate UUIDs
List<String> uuids = Stream.generate(() -> UUID.randomUUID().toString())
    .limit(100)
    .collect(Collectors.toList());
```

Real-World Use Case

```
// Generating test data for load testing
List<Employee> testEmployees = Stream.generate(() -> new Employee(
        "Emp-" + ThreadLocalRandom.current().nextInt(10000),
        departments[ThreadLocalRandom.current().nextInt(departments.length)],
        30000 + ThreadLocalRandom.current().nextInt(70000)
    ))
```

```
    .limit(10000)
    .collect(Collectors.toList());
```

> ***Interview Discussion:*** `Stream.generate()` *creates an* ***unordered*** *infinite stream. It has no concept of previous elements. For ordered sequences, use* `Stream.iterate()`.

3.7 Stream.iterate()

Syntax & Explanation

Creates an infinite ordered stream by iteratively applying a function to a seed value.

```
// Java 8: Two-argument iterate (infinite, must limit)
Stream<Integer> powers = Stream.iterate(1, n -> n * 2);
// 1, 2, 4, 8, 16, 32, ...

List<Integer> firstTen = Stream.iterate(1, n -> n * 2)
    .limit(10)
    .collect(Collectors.toList());

// Java 9+: Three-argument iterate (with predicate -- like a for loop)
Stream<Integer> range = Stream.iterate(1, n -> n <= 100, n -> n + 1);
// Equivalent to: for (int n = 1; n <= 100; n++)
```

Real-World Use Case

```
// Generating date ranges for a report
List<LocalDate> last30Days = Stream.iterate(
        LocalDate.now(),
        date -> date.isAfter(LocalDate.now().minusDays(30)),
        date -> date.minusDays(1)
    )
    .collect(Collectors.toList());

// Fibonacci sequence
Stream.iterate(new long[]{0, 1}, f -> new long[]{f[1], f[0] + f[1]})
    .limit(20)
```

```
    .map(f -> f[0])
    .forEach(System.out::println);
```

> ***Interview Tip:*** `Stream.iterate()` *differs from* `Stream.generate()` *in that each element depends on the previous one. The Java 9 version with a predicate replaces many* `for` *loop patterns.*

Stream Creation Summary Table

Method	Source	Finite/Infinite	Returns	Use Case
`collection.stream()`	Collection	Finite	`Stream<T>`	Most common -- process lists/sets
`collection.parallelStream()`	Collection	Finite	`Stream<T>`	Large datasets, CPU-bound work
`Stream.of(...)`	Explicit values	Finite	`Stream<T>`	Small, known element sets
`Arrays.stream(arr)`	Array	Finite	`Stream<T>` / `IntStream`	Process arrays
`Files.lines(path)`	File	Finite	`Stream<String>`	File processing, log analysis
`Stream.generate(supplier)`	Supplier function	Infinite	`Stream<T>`	Random data, test data generation
`Stream.iterate(seed, fn)`	Seed + unary operator	Infinite	`Stream<T>`	Sequences, ranges, progressions

---

Part 4: All Intermediate Stream Operations

> ***Key Rule:*** *Intermediate operations are LAZY -- they are not executed until a terminal operation is invoked. They always return a new* `Stream`*, allowing method chaining.*

4.1 map()

What It Does

Transforms each element in the stream by applying a function. The function takes one element and returns one transformed element. The stream size remains the same.

Method Signature

```
<R> Stream<R> map(Function<? super T, ? extends R> mapper)
```

When to Use

- Converting objects from one type to another (Entity -> DTO)
- Extracting a field from objects (Employee -> name)
- Applying a transformation to each element

When NOT to Use

- When the mapping function returns a collection and you want to flatten it (use `flatMap()`)
- When you just need side effects without transformation (use `forEach()`)

Time Complexity

O(n) -- applies the function once per element.

Real-World Example

```
// Entity to DTO conversion in a REST API service layer
public List<EmployeeDTO> getAllEmployees() {
    return employeeRepository.findAll().stream()
        .map(entity -> new EmployeeDTO(
            entity.getId(),
```

```
            entity.getFirstName() + " " + entity.getLastName(),
            entity.getDepartment().getName(),
            entity.getSalary()
        ))
        .collect(Collectors.toList());
}
```

Code Example

```
// Transform strings to their lengths
List<Integer> lengths = List.of("Java", "Stream", "API").stream()
    .map(String::length)
    .collect(Collectors.toList());
// Output: [4, 6, 3]

// Convert to uppercase
List<String> upper = List.of("hello", "world").stream()
    .map(String::toUpperCase)
    .collect(Collectors.toList());
// Output: ["HELLO", "WORLD"]
```

> ***Interview Question:*** *What is the difference between* `map()` *and* `flatMap()`*?* `map()` *performs a one-to-one transformation (each element maps to exactly one result).* `flatMap()` *performs a one-to-many transformation and flattens the results (each element can map to zero or more results, and the resulting streams are merged into one).*

4.2 flatMap()

What It Does

Transforms each element into a stream of elements, then **flattens** all the resulting streams into a single stream. Used when each element maps to multiple elements.

Method Signature

```
<R> Stream<R> flatMap(Function<? super T, ? extends Stream<? extends R>>
```

```
mapper)
```

When to Use

- Flattening nested collections (List of Lists -> single List)
- When `map()` would produce `Stream<Stream<T>>` and you want `Stream<T>`
- Processing nested data structures

When NOT to Use

- Simple one-to-one transformations (use `map()`)
- When you don't need to flatten (use `map()`)

Time Complexity

O(n - m) where n is the number of elements and m is the average size of each sub-stream.

Real-World Example

```
// Flatten orders from multiple customers
List<Order> allOrders = customers.stream()
    .flatMap(customer -> customer.getOrders().stream())
    .collect(Collectors.toList());

// Extract all tags from blog posts
List<String> allTags = blogPosts.stream()
    .flatMap(post -> post.getTags().stream())
    .distinct()
    .sorted()
    .collect(Collectors.toList());
```

Code Example

```
// Flatten nested lists
List<List<Integer>> nested = List.of(
    List.of(1, 2, 3),
    List.of(4, 5),
    List.of(6, 7, 8, 9)
);

List<Integer> flat = nested.stream()
```

```
    .flatMap(Collection::stream)
    .collect(Collectors.toList());
// Output: [1, 2, 3, 4, 5, 6, 7, 8, 9]

// Split sentences into words
List<String> sentences = List.of("Hello World", "Java Streams");
List<String> words = sentences.stream()
    .flatMap(sentence -> Arrays.stream(sentence.split(" ")))
    .collect(Collectors.toList());
// Output: ["Hello", "World", "Java", "Streams"]
```

Visual: map() vs flatMap()

```
map() -- One-to-One:
  [A, B, C]  ->  map(x -> transform(x))  ->  [A', B', C']
  Size stays the same

flatMap() -- One-to-Many (then flatten):
  [A, B]  ->  flatMap(x -> Stream.of(x1, x2))  ->  [A1, A2, B1, B2]
  Size may change

Example:
  map:     ["Hello World", "Java"]  -> map(s -> s.split(" "))    ->
[["Hello","World"], ["Java"]]
           Result: Stream<String[]>   NOT what we want!

  flatMap: ["Hello World", "Java"]  -> flatMap(s -> Arrays.stream(s.split(" ")))
           -> ["Hello", "World", "Java"]
           Result: Stream<String>   Flattened! [ok]
```

> ***Interview Question:*** *Can you explain a scenario where using* `map()` *would be incorrect and you must use* `flatMap()`*? When each element maps to a collection. E.g., getting all line items from a list of orders.* `map()` *would give* `Stream<List<LineItem>>`*, but* `flatMap()` *gives* `Stream<LineItem>`*.*

4.3 filter()

What It Does

Returns a stream containing only the elements that match a given predicate. Elements for which the predicate returns `false` are excluded.

Method Signature

```
Stream<T> filter(Predicate<? super T> predicate)
```

When to Use

- Removing unwanted elements from a dataset
- Applying business rules to exclude records
- Searching for elements matching criteria

When NOT to Use

- When you need the index of elements (use a traditional loop)
- When filtering logic has side effects (violates functional principles)

Time Complexity

O(n) -- predicate is evaluated once per element.

Real-World Example

```
// Filter active premium customers with outstanding balance
List<Customer> targetCustomers = customerRepository.findAll().stream()
    .filter(Customer::isActive)
    .filter(c -> c.getMembershipType() == MembershipType.PREMIUM)
    .filter(c -> c.getOutstandingBalance().compareTo(BigDecimal.ZERO) > 0)
    .sorted(Comparator.comparing(Customer::getOutstandingBalance).reversed())
    .collect(Collectors.toList());
```

Code Example

```
// Filter even numbers
List<Integer> evens = List.of(1, 2, 3, 4, 5, 6).stream()
    .filter(n -> n % 2 == 0)
    .collect(Collectors.toList());
// Output: [2, 4, 6]
```

```java
// Filter non-null and non-empty strings
List<String> valid = Arrays.asList("Hello", null, "", "World", null).stream()
    .filter(Objects::nonNull)
    .filter(s -> !s.isEmpty())
    .collect(Collectors.toList());
// Output: ["Hello", "World"]

// Chaining multiple filters vs single complex filter
// Option 1: Multiple filters (more readable)
stream.filter(e -> e.getSalary() > 50000)
      .filter(e -> e.getAge() > 25)
      .filter(e -> "IT".equals(e.getDepartment()));

// Option 2: Single combined filter (slightly more performant)
stream.filter(e -> e.getSalary() > 50000 && e.getAge() > 25 &&
"IT".equals(e.getDepartment()));
```

> ***Interview Question:*** *Is it better to use multiple* `filter()` *calls or one combined* `filter()`*? Multiple filters are more readable and maintainable. A single filter avoids creating intermediate stream objects. For most cases, readability wins. The JVM's loop fusion optimization minimizes the performance difference.*

4.4 distinct()

What It Does

Returns a stream with duplicate elements removed. Uses `equals()` and `hashCode()` to determine uniqueness.

Method Signature

```java
Stream<T> distinct()
```

When to Use

- Removing duplicate entries from data
- Getting unique values from a collection

When NOT to Use

- When objects don't properly implement `equals()` and `hashCode()`
- When you need to control which duplicate is kept (use `Collectors.toMap()` with merge function)

Time Complexity

O(n) on average -- uses a `LinkedHashSet` internally.

Code Example

```
// Remove duplicate integers
List<Integer> unique = List.of(1, 2, 2, 3, 3, 3, 4).stream()
    .distinct()
    .collect(Collectors.toList());
// Output: [1, 2, 3, 4]

// Get unique departments
List<String> departments = employees.stream()
    .map(Employee::getDepartment)
    .distinct()
    .sorted()
    .collect(Collectors.toList());
```

> ***Interview Question:*** *What happens if you call* `distinct()` *on objects that don't override* `equals()` *and* `hashCode()`*? It uses the default* `Object.equals()` *which compares references (memory addresses). Two different objects with the same field values will NOT be considered duplicates. Always override* `equals()` *and* `hashCode()` *for custom objects used with* `distinct()`*.*

4.5 sorted()

What It Does

Returns a stream with elements sorted. Can use natural ordering or a custom `Comparator`.

Method Signature

```
Stream<T> sorted()                          // Natural ordering (Comparable)
Stream<T> sorted(Comparator<? super T> comparator)  // Custom ordering
```

When to Use

- Ordering results before returning to the client
- Finding top-N elements (combine with `limit()`)
- Preparing data for display

When NOT to Use

- On infinite streams (will never terminate!)
- When order doesn't matter (unnecessary processing)
- On parallel streams with large datasets (sorting requires synchronization)

Time Complexity

O(n log n) -- uses a modified merge sort internally.

Code Example

```
// Natural ordering
List<String> sorted = List.of("Charlie", "Alice", "Bob").stream()
    .sorted()
    .collect(Collectors.toList());
// Output: [Alice, Bob, Charlie]

// Custom comparator -- sort by salary descending
List<Employee> topPaid = employees.stream()
    .sorted(Comparator.comparing(Employee::getSalary).reversed())
    .limit(5)
    .collect(Collectors.toList());

// Multi-field sorting
List<Employee> sorted = employees.stream()
    .sorted(Comparator.comparing(Employee::getDepartment)
            .thenComparing(Employee::getSalary).reversed()
            .thenComparing(Employee::getName))
    .collect(Collectors.toList());
```

> ***Interview Question:*** *Can you sort a stream of custom objects without implementing Comparable? Yes, by passing a Comparator to* `sorted()` *. E.g.,* `stream.sorted(Comparator.comparing(Employee::getSalary))` *. If you use* `sorted()` *without arguments on objects that don't implement Comparable, you'll get a* `ClassCastException` *.*

4.6 peek()

What It Does

Performs an action on each element as it passes through the pipeline, **without modifying the stream**. Primarily used for debugging.

Method Signature

```
Stream<T> peek(Consumer<? super T> action)
```

When to Use

- Debugging stream pipelines (logging element values)
- Adding diagnostic logging in production (sparingly)

When NOT to Use

- As a replacement for `forEach()` (it's an intermediate operation, not terminal)
- To modify elements (violates the functional contract)
- In production code for business logic (side effects in a pipeline make code harder to reason about)

Code Example

```
// Debugging: see what passes through each stage
List<String> result = List.of("one", "two", "three", "four").stream()
    .filter(s -> s.length() > 3)
    .peek(s -> System.out.println("After filter: " + s))
    .map(String::toUpperCase)
    .peek(s -> System.out.println("After map: " + s))
    .collect(Collectors.toList());
```

```
// Output:
//   After filter: three
//   After map: THREE
//   After filter: four
//   After map: FOUR
```

> ***Interview Question:*** *Is* `peek()` *guaranteed to execute? No! Because streams are lazy, if the terminal operation doesn't consume all elements (e.g.,* `findFirst()` *),* `peek()` *won't be called for unconsumed elements. Also, the JVM may optimize away* `peek()` *calls in some scenarios.*

4.7 limit()

What It Does

Returns a stream containing at most the first `n` elements. If the stream has fewer than `n` elements, all elements are returned.

Method Signature

```
Stream<T> limit(long maxSize)
```

When to Use

- Implementing pagination
- Getting top N results
- Truncating infinite streams

When NOT to Use

- When you need elements from the middle or end (combine with `skip()`)

Time Complexity

O(n) in the worst case, but short-circuits -- stops processing after `n` elements.

Code Example

```
// Get top 3 highest-paid employees
List<Employee> top3 = employees.stream()
    .sorted(Comparator.comparing(Employee::getSalary).reversed())
    .limit(3)
    .collect(Collectors.toList());

// Truncate infinite stream
List<Integer> first10Squares = Stream.iterate(1, n -> n + 1)
    .map(n -> n * n)
    .limit(10)
    .collect(Collectors.toList());
// Output: [1, 4, 9, 16, 25, 36, 49, 64, 81, 100]
```

4.8 skip()

What It Does

Returns a stream that skips the first `n` elements. If the stream has fewer than `n` elements, an empty stream is returned.

Method Signature

```
Stream<T> skip(long n)
```

When to Use

- Implementing pagination (combine with `limit()`)
- Skipping header rows in file processing

When NOT to Use

- With unordered parallel streams (elements skipped are unpredictable)

Code Example

```
// Pagination: Page 3, 10 items per page
int page = 3, size = 10;
List<Employee> page3 = employees.stream()
    .skip((long)(page - 1) * size)  // Skip first 20
```

```
    .limit(size)                    // Take next 10
    .collect(Collectors.toList());

// Skip header row in CSV processing
try (Stream<String> lines = Files.lines(Paths.get("data.csv"))) {
    List<String[]> data = lines
        .skip(1)  // Skip header row
        .map(line -> line.split(","))
        .collect(Collectors.toList());
}
```

Intermediate Operations Summary Table

Operation	Purpose	Returns	Stateful?	Short-circuits?
`map()`	Transform elements	`Stream<R>`	No	No
`flatMap()`	Transform & flatten	`Stream<R>`	No	No
`filter()`	Keep matching elements	`Stream<T>`	No	No
`distinct()`	Remove duplicates	`Stream<T>`	Yes	No
`sorted()`	Sort elements	`Stream<T>`	Yes	No
`peek()`	Debug/inspect	`Stream<T>`	No	No
`limit()`	Truncate stream	`Stream<T>`	Yes	Yes
`skip()`	Skip first N	`Stream<T>`	Yes	No

> ***Stateful*** *operations need to see all elements before producing output (like sorting requires all elements).* ***Stateless*** *operations process each element independently.*

Part 5: All Terminal Stream Operations

> ***Key Rule:*** *Terminal operations trigger the execution of the entire stream pipeline. After a terminal operation, the stream is consumed and cannot be reused.*

5.1 forEach()

Syntax & How It Works

```
void forEach(Consumer<? super T> action)
```

Performs an action on each element. It does NOT return a value. **Order is not guaranteed** in parallel streams.

Use Cases

```
// Print all elements
employees.stream()
    .forEach(e -> System.out.println(e.getName()));

// Using method reference
employees.stream().forEach(System.out::println);

// Sending notifications to each user
activeUsers.stream()
    .filter(User::isSubscribed)
    .forEach(user -> notificationService.send(user, message));
```

Performance Notes

- `forEach()` is a terminal operation -- it triggers pipeline execution
- For parallel streams, use `forEachOrdered()` if order matters
- Avoid using `forEach()` to add elements to external collections (use `collect()` instead)

forEachOrdered() for Parallel Streams

```java
// forEach on parallel stream -- order NOT guaranteed
List.of(1, 2, 3, 4, 5).parallelStream()
    .forEach(System.out::print);       // Output: 35214 (random order)

// forEachOrdered on parallel stream -- order IS guaranteed
List.of(1, 2, 3, 4, 5).parallelStream()
    .forEachOrdered(System.out::print); // Output: 12345 (encounter order)
```

5.2 collect()

Syntax & How It Works

```java
<R, A> R collect(Collector<? super T, A, R> collector)
<R> R collect(Supplier<R> supplier, BiConsumer<R, ? super T> accumulator,
BiConsumer<R, R> combiner)
```

The most versatile terminal operation. Transforms stream elements into a different form -- typically a collection, string, or aggregated value. Uses the **Mutable Reduction** pattern.

Use Cases

```java
// Collect to List
List<String> names = stream.collect(Collectors.toList());

// Collect to Set (removes duplicates)
Set<String> uniqueNames = stream.collect(Collectors.toSet());

// Collect to Map
Map<Long, String> idToName = employees.stream()
    .collect(Collectors.toMap(Employee::getId, Employee::getName));

// Collect to String
String csv = names.stream().collect(Collectors.joining(", "));
```

> *See **Part 6: Collectors Deep Dive** for comprehensive `collect()` patterns.*

5.3 reduce()

Syntax & How It Works

```
// Variant 1: With identity
T reduce(T identity, BinaryOperator<T> accumulator)

// Variant 2: Without identity (returns Optional)
Optional<T> reduce(BinaryOperator<T> accumulator)

// Variant 3: With identity, accumulator, and combiner (for parallel)
<U> U reduce(U identity, BiFunction<U, ? super T, U> accumulator,
BinaryOperator<U> combiner)
```

Combines all elements into a single result by repeatedly applying a binary operation.

How It Works Internally

```
reduce(0, (a, b) -> a + b)  on  [1, 2, 3, 4]

Step 1: result = 0 (identity)
Step 2: result = 0 + 1 = 1
Step 3: result = 1 + 2 = 3
Step 4: result = 3 + 3 = 6
Step 5: result = 6 + 4 = 10
Final:  10
```

Use Cases

```
// Sum of numbers
int sum = List.of(1, 2, 3, 4, 5).stream()
    .reduce(0, Integer::sum);           // 15

// Without identity -- returns Optional (handles empty stream)
Optional<Integer> sum = List.of(1, 2, 3).stream()
    .reduce(Integer::sum);              // Optional[6]

// Find maximum salary
Optional<Integer> maxSalary = employees.stream()
    .map(Employee::getSalary)
```

```
    .reduce(Integer::max);

// Concatenate strings
String combined = List.of("Hello", " ", "World").stream()
    .reduce("", String::concat);        // "Hello World"

// Complex reduction: Total revenue
BigDecimal totalRevenue = orders.stream()
    .map(Order::getAmount)
    .reduce(BigDecimal.ZERO, BigDecimal::add);
```

> ***Interview Question:*** *Difference between* `reduce()` *and* `collect()`*?* `reduce()` *produces an* ***immutable*** *result by combining elements (good for sum, min, max, string concat).* `collect()` *produces a* ***mutable*** *result container (good for building collections, StringBuilder). Use* `collect()` *for mutable reductions (lists, maps) and* `reduce()` *for value computations.*

5.4 count()

Syntax & How It Works

```
long count()
```

Returns the number of elements in the stream. This is a terminal operation.

```
long total = employees.stream().count();

long activeCount = employees.stream()
    .filter(Employee::isActive)
    .count();

// Counting unique departments
long deptCount = employees.stream()
    .map(Employee::getDepartment)
    .distinct()
    .count();
```

Performance Notes

For simple counts on collections, `collection.size()` is O(1) and more efficient than `stream().count()`.

5.5 min() and max()

Syntax & How It Works

```
Optional<T> min(Comparator<? super T> comparator)
Optional<T> max(Comparator<? super T> comparator)
```

Returns the minimum or maximum element according to the provided comparator. Returns `Optional.empty()` if the stream is empty.

```
// Find youngest employee
Optional<Employee> youngest = employees.stream()
    .min(Comparator.comparing(Employee::getAge));

// Find highest salary
Optional<Integer> maxSalary = employees.stream()
    .map(Employee::getSalary)
    .max(Comparator.naturalOrder());

// Find longest string
Optional<String> longest = List.of("Java", "Streams", "API").stream()
    .max(Comparator.comparingInt(String::length));
// Output: Optional[Streams]
```

5.6 findFirst() and findAny()

Syntax & How It Works

```
Optional<T> findFirst()   // Returns first element in encounter order
Optional<T> findAny()     // Returns any element (optimized for parallel)
```

Both are **short-circuiting** terminal operations -- they don't need to process the entire stream.

```java
// Find first employee with salary > 100000
Optional<Employee> first = employees.stream()
    .filter(e -> e.getSalary() > 100000)
    .findFirst();

// findAny is better for parallel streams
Optional<Employee> any = employees.parallelStream()
    .filter(e -> e.getSalary() > 100000)
    .findAny();

// Using with orElse
String name = employees.stream()
    .filter(e -> e.getDepartment().equals("Engineering"))
    .map(Employee::getName)
    .findFirst()
    .orElse("Not Found");
```

Interview Question: *When would you use* `findAny()` *over* `findFirst()`*? In parallel streams,* `findFirst()` *is constrained to return the first element in encounter order, which reduces parallelism.* `findAny()` *can return whichever element is found first by any thread, providing better parallel performance. In sequential streams, both behave identically.*

5.7 anyMatch(), allMatch(), noneMatch()

Syntax & How It Works

```java
boolean anyMatch(Predicate<? super T> predicate)   // true if ANY element
matches
boolean allMatch(Predicate<? super T> predicate)    // true if ALL elements
match
boolean noneMatch(Predicate<? super T> predicate)   // true if NO elements
match
```

All three are **short-circuiting** -- they return as soon as the answer is determined.

```java
List<Integer> numbers = List.of(1, 2, 3, 4, 5);
```

```java
boolean hasEven = numbers.stream().anyMatch(n -> n % 2 == 0);     // true
boolean allPositive = numbers.stream().allMatch(n -> n > 0);       // true
boolean noneNegative = numbers.stream().noneMatch(n -> n < 0);     // true

// Real-world: Check if any order is overdue
boolean hasOverdue = orders.stream()
    .anyMatch(order -> order.getDueDate().isBefore(LocalDate.now())
                    && order.getStatus() != OrderStatus.COMPLETED);

// Check if all employees have valid email
boolean allValid = employees.stream()
    .allMatch(e -> e.getEmail() != null && e.getEmail().contains("@"));
```

Short-Circuit Behavior

```
anyMatch: stops at FIRST true  -> [false, false, TRUE, ...] -> stops at 3rd
element
allMatch: stops at FIRST false -> [true, true, FALSE, ...] -> stops at 3rd
element
noneMatch: stops at FIRST true -> [false, false, TRUE, ...] -> stops at 3rd
element
```

5.8 toArray()

Syntax & How It Works

```java
Object[] toArray()                              // Returns Object[]
<A> A[] toArray(IntFunction<A[]> generator)     // Returns typed array
```

Converts stream elements to an array.

```java
// To Object array
Object[] arr = List.of("A", "B", "C").stream().toArray();

// To typed array (preferred)
String[] names = employees.stream()
    .map(Employee::getName)
    .toArray(String[]::new);
```

```
// To int array
int[] numbers = IntStream.rangeClosed(1, 10).toArray();
```

Terminal Operations Summary Table

Operation	Returns	Short-Circuit?	Purpose
`forEach()`	`void`	No	Side effects (print, send, save)
`collect()`	`R`	No	Accumulate into collection/result
`reduce()`	`T` / `Optional<T>`	No	Combine elements into single value
`count()`	`long`	No	Count elements
`min()`	`Optional<T>`	No	Find minimum element
`max()`	`Optional<T>`	No	Find maximum element
`findFirst()`	`Optional<T>`	Yes	Find first matching element
`findAny()`	`Optional<T>`	Yes	Find any matching element
`anyMatch()`	`boolean`	Yes	Check if any element matches
`allMatch()`	`boolean`	Yes	Check if all elements match
`noneMatch()`	`boolean`	Yes	Check if no elements match
`toArray()`	`T[]`	No	Convert to array

Part 6: Collectors Deep Dive

6.1 Collectors.toList()

Collects stream elements into a `List`. The most commonly used collector.

```
List<String> names = employees.stream()
    .map(Employee::getName)
    .collect(Collectors.toList());

// Java 16+: Shorthand using Stream.toList() -- returns unmodifiable list
List<String> names = employees.stream()
    .map(Employee::getName)
    .toList();
```

6.2 Collectors.toSet()

Collects into a `Set`, automatically removing duplicates.

```
Set<String> departments = employees.stream()
    .map(Employee::getDepartment)
    .collect(Collectors.toSet());
// Duplicates removed automatically
```

6.3 Collectors.toMap()

Collects into a `Map` with key and value mapping functions.

```
// Basic: Employee ID -> Employee Name
Map<Long, String> idToName = employees.stream()
    .collect(Collectors.toMap(Employee::getId, Employee::getName));

// With value being the object itself
Map<Long, Employee> idToEmployee = employees.stream()
    .collect(Collectors.toMap(Employee::getId, Function.identity()));
```

```java
// Handle duplicate keys with merge function
Map<String, Integer> deptToMaxSalary = employees.stream()
    .collect(Collectors.toMap(
        Employee::getDepartment,
        Employee::getSalary,
        Integer::max  // If duplicate key, keep the higher salary
    ));

// Specify map implementation
Map<String, Employee> sortedMap = employees.stream()
    .collect(Collectors.toMap(
        Employee::getName,
        Function.identity(),
        (e1, e2) -> e1,      // Merge function for duplicates
        TreeMap::new           // Use TreeMap for sorted keys
    ));
```

> ***Interview Tip:*** *Without a merge function,* `toMap()` *throws* `IllegalStateException` *if duplicate keys exist. Always provide a merge function when keys might not be unique.*

6.4 Collectors.groupingBy()

Groups elements by a classifier function. Returns a `Map<K, List<T>>`.

```java
// Group employees by department
Map<String, List<Employee>> byDept = employees.stream()
    .collect(Collectors.groupingBy(Employee::getDepartment));
// Output: {IT=[emp1, emp2], HR=[emp3], Finance=[emp4, emp5]}

// Group by department and count
Map<String, Long> deptCount = employees.stream()
    .collect(Collectors.groupingBy(
        Employee::getDepartment,
        Collectors.counting()
    ));
// Output: {IT=2, HR=1, Finance=2}

// Group by department and get average salary
Map<String, Double> avgSalaryByDept = employees.stream()
    .collect(Collectors.groupingBy(
        Employee::getDepartment,
        Collectors.averagingDouble(Employee::getSalary)
```

```
    ));

// Group by department, then get names
Map<String, List<String>> namesByDept = employees.stream()
    .collect(Collectors.groupingBy(
        Employee::getDepartment,
        Collectors.mapping(Employee::getName, Collectors.toList())
    ));

// Multi-level grouping: Department -> City -> List<Employee>
Map<String, Map<String, List<Employee>>> multiLevel = employees.stream()
    .collect(Collectors.groupingBy(
        Employee::getDepartment,
        Collectors.groupingBy(Employee::getCity)
    ));

// Group by department and find max salary employee
Map<String, Optional<Employee>> highestPaid = employees.stream()
    .collect(Collectors.groupingBy(
        Employee::getDepartment,
        Collectors.maxBy(Comparator.comparing(Employee::getSalary))
    ));
```

Real-World Example: Analytics Dashboard

```
// Group orders by status and calculate total amounts
Map<OrderStatus, BigDecimal> revenueByStatus = orders.stream()
    .collect(Collectors.groupingBy(
        Order::getStatus,
        Collectors.reducing(
            BigDecimal.ZERO,
            Order::getAmount,
            BigDecimal::add
        )
    ));
```

6.5 Collectors.partitioningBy()

Splits elements into two groups based on a predicate. Returns `Map<Boolean, List<T>>`.

```java
// Partition employees by salary threshold
Map<Boolean, List<Employee>> partitioned = employees.stream()
    .collect(Collectors.partitioningBy(e -> e.getSalary() > 50000));

List<Employee> highPaid = partitioned.get(true);   // Salary > 50000
List<Employee> lowPaid = partitioned.get(false);    // Salary <= 50000

// Partition with downstream collector
Map<Boolean, Long> counts = employees.stream()
    .collect(Collectors.partitioningBy(
        e -> e.getSalary() > 50000,
        Collectors.counting()
    ));

// Partition even/odd numbers
Map<Boolean, List<Integer>> evenOdd = numbers.stream()
    .collect(Collectors.partitioningBy(n -> n % 2 == 0));
```

Interview Tip: `partitioningBy()` *always returns a map with both* `true` *and* `false` *keys (even if one group is empty).* `groupingBy()` *only includes keys that have at least one element.*

6.6 Collectors.joining()

Concatenates stream elements into a single `String`.

```java
// Simple join
String names = List.of("Alice", "Bob", "Charlie").stream()
    .collect(Collectors.joining());
// Output: "AliceBobCharlie"

// Join with delimiter
String csv = List.of("Alice", "Bob", "Charlie").stream()
    .collect(Collectors.joining(", "));
// Output: "Alice, Bob, Charlie"

// Join with delimiter, prefix, and suffix
String json = List.of("Alice", "Bob", "Charlie").stream()
    .collect(Collectors.joining("\", \"", "[\"", "\"]"));
// Output: ["Alice", "Bob", "Charlie"]
```

```
// Real-world: Build SQL IN clause
String inClause = ids.stream()
    .map(String::valueOf)
    .collect(Collectors.joining(", ", "SELECT * FROM users WHERE id IN (",
")"));
```

6.7 Collectors.summarizingInt() / summarizingDouble() / summarizingLong()

Provides summary statistics (count, sum, min, max, average) in one pass.

```
IntSummaryStatistics stats = employees.stream()
    .collect(Collectors.summarizingInt(Employee::getSalary));

System.out.println("Count:   " + stats.getCount());    // 5
System.out.println("Sum:     " + stats.getSum());       // 275000
System.out.println("Min:     " + stats.getMin());       // 35000
System.out.println("Max:     " + stats.getMax());       // 85000
System.out.println("Average: " + stats.getAverage());   // 55000.0

// For double values
DoubleSummaryStatistics doubleStats = products.stream()
    .collect(Collectors.summarizingDouble(Product::getPrice));
```

6.8 Collectors.mapping()

Adapts a collector to accept elements of a different type by applying a mapping function first.

```
// Get names grouped by department
Map<String, List<String>> namesByDept = employees.stream()
    .collect(Collectors.groupingBy(
        Employee::getDepartment,
        Collectors.mapping(Employee::getName, Collectors.toList())
    ));

// Get comma-separated names by department
Map<String, String> nameStringByDept = employees.stream()
    .collect(Collectors.groupingBy(
        Employee::getDepartment,
```

```
        Collectors.mapping(
            Employee::getName,
            Collectors.joining(", ")
        )
    ));
```

Collectors Summary Table

Collector	Returns	Purpose
`toList()`	`List<T>`	Collect to list
`toSet()`	`Set<T>`	Collect to set (unique elements)
`toMap()`	`Map<K,V>`	Collect to key-value map
`groupingBy()`	`Map<K, List<T>>`	Group elements by classifier
`partitioningBy()`	`Map<Boolean, List<T>>`	Split into two groups
`joining()`	`String`	Concatenate elements
`summarizingInt()`	`IntSummaryStatistics`	Compute statistics
`mapping()`	Depends on downstream	Transform before collecting
`counting()`	`Long`	Count elements in group
`reducing()`	`T` / `Optional<T>`	Custom reduction

Part 7: Functional Interfaces (Complete Guide)

> *A **Functional Interface** is an interface with exactly ONE abstract method. It can have multiple default and static methods. The `@FunctionalInterface` annotation is optional but recommended.*

7.1 Predicate

Definition and Method Signature

```java
@FunctionalInterface
public interface Predicate<T> {
    boolean test(T t);

    // Default methods
    default Predicate<T> and(Predicate<? super T> other);
    default Predicate<T> or(Predicate<? super T> other);
    default Predicate<T> negate();

    // Static method
    static <T> Predicate<T> isEqual(Object targetRef);
}
```

Where It Is Used

- `Stream.filter(Predicate)`
- `Collection.removeIf(Predicate)`
- `Optional.filter(Predicate)`

Real Examples

```java
// Simple predicate
Predicate<Integer> isPositive = n -> n > 0;
Predicate<String> isNotEmpty = s -> s != null && !s.isEmpty();

// Combining predicates
Predicate<Employee> isHighPaid = e -> e.getSalary() > 80000;
Predicate<Employee> isIT = e -> "IT".equals(e.getDepartment());
Predicate<Employee> isHighPaidIT = isHighPaid.and(isIT);
```

```
// Using in stream
employees.stream()
    .filter(isHighPaidIT)
    .collect(Collectors.toList());

// Negation
Predicate<Employee> isNotIT = isIT.negate();
```

> ***Interview Question:*** *How do you compose multiple Predicates? Use* `.and()`, `.or()`, *and* `.negate()` *for logical AND, OR, and NOT operations. E.g.,* `predicate1.and(predicate2).or(predicate3)`.

7.2 Function<T, R>

Definition and Method Signature

```
@FunctionalInterface
public interface Function<T, R> {
    R apply(T t);

    default <V> Function<V, R> compose(Function<? super V, ? extends T>
before);
    default <V> Function<T, V> andThen(Function<? super R, ? extends V> after);
    static <T> Function<T, T> identity();
}
```

Where It Is Used

- `Stream.map(Function)`
- `Map.computeIfAbsent(key, Function)`
- `Optional.map(Function)`

Real Examples

```
// Transform Employee to DTO
Function<Employee, EmployeeDTO> toDTO = emp -> new EmployeeDTO(
    emp.getId(), emp.getName(), emp.getSalary()
```

```
);

// Function composition
Function<String, String> trim = String::trim;
Function<String, String> upperCase = String::toUpperCase;
Function<String, String> trimAndUpper = trim.andThen(upperCase);

String result = trimAndUpper.apply("  hello  ");  // "HELLO"

// Using identity
Map<Long, Employee> map = employees.stream()
    .collect(Collectors.toMap(Employee::getId, Function.identity()));
```

> ***Interview Question:*** *What does* `Function.identity()` *do? Returns a function that always returns its input argument unchanged.* `Function.identity()` *is equivalent to* `t -> t`*. It's used when you need to pass the element itself as a value (e.g., in* `toMap()`*).*

7.3 Consumer

Definition and Method Signature

```
@FunctionalInterface
public interface Consumer<T> {
    void accept(T t);

    default Consumer<T> andThen(Consumer<? super T> after);
}
```

Where It Is Used

- `Stream.forEach(Consumer)`
- `Stream.peek(Consumer)`
- `Iterable.forEach(Consumer)`
- `Optional.ifPresent(Consumer)`

Real Examples

```
Consumer<Employee> printEmployee = e -> System.out.println(e.getName());
Consumer<Employee> saveToDb = e -> employeeRepository.save(e);
Consumer<Employee> sendEmail = e -> emailService.sendWelcome(e.getEmail());

// Chain consumers
Consumer<Employee> onboard =
saveToDb.andThen(sendEmail).andThen(printEmployee);

// Apply to each new employee
newEmployees.forEach(onboard);
```

7.4 Supplier

Definition and Method Signature

```
@FunctionalInterface
public interface Supplier<T> {
    T get();
}
```

Where It Is Used

- `Stream.generate(Supplier)`
- `Optional.orElseGet(Supplier)`
- `Objects.requireNonNull(obj, Supplier<String>)`
- Factory patterns

Real Examples

```
// Lazy initialization
Supplier<Connection> connectionSupplier = () -> dataSource.getConnection();
Supplier<LocalDateTime> now = LocalDateTime::now;
Supplier<UUID> idGenerator = UUID::randomUUID;

// Used with Optional (lazy evaluation)
Optional<Employee> emp = findById(id);
Employee result = emp.orElseGet(() -> createDefaultEmployee());
// The default employee is only created if emp is empty
```

```
// Stream generation
List<String> ids = Stream.generate(() -> UUID.randomUUID().toString())
    .limit(100)
    .collect(Collectors.toList());
```

Interview Question: *What's the difference between* `orElse()` *and* `orElseGet()`*?* `orElse(value)` *always evaluates the argument (even if Optional has a value).* `orElseGet(Supplier)` *only evaluates the Supplier if the Optional is empty. Use* `orElseGet()` *when the default value is expensive to create.*

7.5 UnaryOperator

Definition and Method Signature

```
@FunctionalInterface
public interface UnaryOperator<T> extends Function<T, T> {
    // Inherits apply(T t) from Function
    static <T> UnaryOperator<T> identity();
}
```

A specialized `Function` where input and output types are the same.

Real Examples

```
UnaryOperator<String> toUpper = String::toUpperCase;
UnaryOperator<Integer> doubleIt = n -> n * 2;

// Used in Stream.iterate
Stream.iterate(1, n -> n * 2)  // UnaryOperator<Integer>
    .limit(10)
    .forEach(System.out::println);

// List.replaceAll
List<String> names = new ArrayList<>(Arrays.asList("alice", "bob"));
names.replaceAll(String::toUpperCase);  // ["ALICE", "BOB"]
```

7.6 BinaryOperator

Definition and Method Signature

```java
@FunctionalInterface
public interface BinaryOperator<T> extends BiFunction<T, T, T> {
    // Inherits apply(T t1, T t2) from BiFunction
    static <T> BinaryOperator<T> minBy(Comparator<? super T> comparator);
    static <T> BinaryOperator<T> maxBy(Comparator<? super T> comparator);
}
```

A specialized `BiFunction` where both inputs and output are the same type.

Real Examples

```java
BinaryOperator<Integer> sum = Integer::sum;
BinaryOperator<Integer> max = Integer::max;
BinaryOperator<String> concat = String::concat;

// Used in reduce()
int total = numbers.stream().reduce(0, Integer::sum);

// BinaryOperator.maxBy
BinaryOperator<Employee> highestPaid =
    BinaryOperator.maxBy(Comparator.comparing(Employee::getSalary));
```

Functional Interfaces Summary Table

Interface	Method	Input	Output	Used In
`Predicate<T>`	`test(T)`	T	boolean	filter, removeIf
`Function<T,R>`	`apply(T)`	T	R	map, computeIfAbsent
`Consumer<T>`	`accept(T)`	T	void	forEach, peek
`Supplier<T>`	`get()`	none	T	generate, orElseGet
`UnaryOperator<T>`	`apply(T)`	T	T	iterate, replaceAll
`BinaryOperator<T>`	`apply(T,T)`	T, T	T	reduce, merge

Primitive Specializations

Generic	int	long	double
`Predicate<T>`	`IntPredicate`	`LongPredicate`	`DoublePredicate`
`Function<T,R>`	`IntFunction<R>`	`LongFunction<R>`	`DoubleFunction<R>`
`Consumer<T>`	`IntConsumer`	`LongConsumer`	`DoubleConsumer`
`Supplier<T>`	`IntSupplier`	`LongSupplier`	`DoubleSupplier`
`UnaryOperator<T>`	`IntUnaryOperator`	`LongUnaryOperator`	`DoubleUnaryOperator`
`BinaryOperator<T>`	`IntBinaryOperator`	`LongBinaryOperator`	`DoubleBinaryOperator`

> *These primitive specializations avoid autoboxing overhead and should be used when working with primitive types in performance-sensitive code.*

Part 8: Method References

8.1 What Are Method References?

Method references are shorthand for lambdas that call a single existing method. They make code cleaner and more readable.

```
Lambda Expression              ->  Method Reference
(s) -> s.toUpperCase()         ->  String::toUpperCase
(s) -> System.out.println(s)   ->  System.out::println
(s) -> Integer.parseInt(s)     ->  Integer::parseInt
() -> new ArrayList<>()        ->  ArrayList::new
```

8.2 Static Method Reference

Syntax: `ClassName::staticMethod`

```java
// Lambda
Function<String, Integer> parse = s -> Integer.parseInt(s);
// Method reference
Function<String, Integer> parse = Integer::parseInt;

// In streams
List<Integer> numbers = List.of("1", "2", "3").stream()
    .map(Integer::parseInt)
    .collect(Collectors.toList());

// Multiple examples
BinaryOperator<Integer> maxOp = Math::max;
Function<Double, Double> sqrt = Math::sqrt;
```

8.3 Instance Method Reference

Two Forms

Form 1: `instance::method` -- Reference to a method of a specific object

```
PrintStream out = System.out;
Consumer<String> printer = out::println;

employees.stream()
    .map(Employee::getName)
    .forEach(System.out::println);
```

Form 2: `ClassName::instanceMethod` -- Reference to an instance method of an arbitrary object

```
Function<String, String> upper = String::toUpperCase;
Function<String, Integer> length = String::length;

List<String> upperNames = names.stream()
    .map(String::toUpperCase)  // same as: s -> s.toUpperCase()
    .collect(Collectors.toList());

// Comparator using method reference
List<String> sorted = names.stream()
    .sorted(String::compareToIgnoreCase)
    .collect(Collectors.toList());
```

8.4 Constructor Reference

Syntax: `ClassName::new`

```
// Lambda
Supplier<List<String>> listFactory = () -> new ArrayList<>();
// Constructor reference
Supplier<List<String>> listFactory = ArrayList::new;

// With parameters
Function<String, Employee> empFactory = Employee::new;

// In streams -- creating objects
List<Employee> employees = names.stream()
    .map(Employee::new)               // calls new Employee(name)
    .collect(Collectors.toList());
```

```
// Array constructor reference
String[] names = stream.toArray(String[]::new);
```

8.5 Method Reference Best Practices

Guideline	Example
Use when lambda calls a single method	`s -> s.length()` -> `String::length`
Don't use when lambda has additional logic	Keep `s -> s.length() > 5` as lambda
Prefer method references for readability	`Employee::getName` over `e -> e.getName()`
Use constructor references for factories	`ArrayList::new` over `() -> new ArrayList<>()`

Method Reference Types Summary

Type	Syntax	Lambda Equivalent	Example
Static method	`ClassName::method`	`(args) -> ClassName.method(args)`	`Integer::parseInt`
Instance (bound)	`instance::method`	`(args) -> instance.method(args)`	`System.out::println`
Instance (unbound)	`ClassName::method`	`(obj, args) -> obj.method(args)`	`String::toUpperCase`
Constructor	`ClassName::new`	`(args) -> new ClassName(args)`	`ArrayList::new`

Part 9: Comparable vs Comparator (Deep Guide)

9.1 Comparable Interface

Definition

`Comparable<T>` defines the **natural ordering** of objects. The class itself decides how it should be compared.

```
public interface Comparable<T> {
    int compareTo(T o);
    // Returns:
    //   negative  -> this < o
    //   zero      -> this == o
    //   positive  -> this > o
}
```

Implementation Example

```
public class Employee implements Comparable<Employee> {
    private String name;
    private int salary;
    private int age;

    @Override
    public int compareTo(Employee other) {
        return Integer.compare(this.salary, other.salary);  // Natural order:
by salary
    }
}

// Usage -- sorting uses compareTo automatically
List<Employee> sorted = employees.stream()
    .sorted()  // Uses compareTo (natural ordering = by salary)
    .collect(Collectors.toList());

Collections.sort(employees);  // Also uses compareTo
```

9.2 Comparator Interface

Definition

`Comparator<T>` defines **custom ordering** externally. The comparison logic is separate from the class.

```
@FunctionalInterface
public interface Comparator<T> {
    int compare(T o1, T o2);

    // Key static/default methods:
    static <T, U extends Comparable<? super U>> Comparator<T>
        comparing(Function<T, U> keyExtractor);
    default Comparator<T> reversed();
    default Comparator<T> thenComparing(Comparator<? super T> other);
    static <T> Comparator<T> naturalOrder();
    static <T> Comparator<T> reverseOrder();
    static <T> Comparator<T> nullsFirst(Comparator<? super T> comparator);
    static <T> Comparator<T> nullsLast(Comparator<? super T> comparator);
}
```

9.3 Comparable vs Comparator Differences

Aspect	Comparable	Comparator
Package	`java.lang`	`java.util`
Method	`compareTo(T o)`	`compare(T o1, T o2)`
Implementation	Inside the class being compared	External class or lambda
Ordering	Defines ONE natural ordering	Defines MULTIPLE custom orderings
Modification	Requires modifying the original class	No modification needed
Number of orderings	Only one per class	Unlimited per class
Functional Interface?	No	Yes

9.4 When to Use Which

Scenario	Use
The class has an obvious, single natural ordering	`Comparable`
You need multiple different orderings	`Comparator`
The class is from a library (you can't modify it)	`Comparator`
Sorting is a primary feature of the domain object	`Comparable`
Ad-hoc sorting in specific use cases	`Comparator`

9.5 Custom Sorting Examples

Sort by Single Field

```
// By salary ascending
employees.stream()
    .sorted(Comparator.comparing(Employee::getSalary))
    .collect(Collectors.toList());

// By salary descending
employees.stream()
    .sorted(Comparator.comparing(Employee::getSalary).reversed())
    .collect(Collectors.toList());

// By name (case-insensitive)
employees.stream()
    .sorted(Comparator.comparing(Employee::getName,
String.CASE_INSENSITIVE_ORDER))
    .collect(Collectors.toList());

// By age ascending (using primitive comparator for performance)
employees.stream()
    .sorted(Comparator.comparingInt(Employee::getAge))
    .collect(Collectors.toList());
```

Multi-Field Sorting

```
// Sort by department, then by salary (descending), then by name
Comparator<Employee> multiSort = Comparator
    .comparing(Employee::getDepartment)
```

```
    .thenComparing(Employee::getSalary, Comparator.reverseOrder())
    .thenComparing(Employee::getName);

List<Employee> sorted = employees.stream()
    .sorted(multiSort)
    .collect(Collectors.toList());
```

Handling Nulls

```
// Nulls first
employees.stream()
    .sorted(Comparator.comparing(Employee::getName,
        Comparator.nullsFirst(Comparator.naturalOrder())))
    .collect(Collectors.toList());

// Nulls last
employees.stream()
    .sorted(Comparator.comparing(Employee::getManager,
        Comparator.nullsLast(Comparator.naturalOrder())))
    .collect(Collectors.toList());
```

Real-World Example: Complex Employee Sorting

```
// Sort employees:
// 1. Active employees first, then inactive
// 2. Within each group, by department alphabetically
// 3. Within each department, by salary descending
// 4. If salary is same, by name alphabetically

Comparator<Employee> complexSort = Comparator
    .comparing(Employee::isActive, Comparator.reverseOrder())
    .thenComparing(Employee::getDepartment)
    .thenComparing(Employee::getSalary, Comparator.reverseOrder())
    .thenComparing(Employee::getName);

List<Employee> result = employees.stream()
    .sorted(complexSort)
    .collect(Collectors.toList());
```

Part 10: Advanced Stream Topics

10.1 Parallel Streams

What Are Parallel Streams?

Parallel streams split the data into multiple chunks and process them concurrently using the **ForkJoinPool**. This can significantly improve performance for CPU-intensive operations on large datasets.

```
Sequential Stream:
  [1, 2, 3, 4, 5, 6, 7, 8]
  Thread 1: 1 -> 2 -> 3 -> 4 -> 5 -> 6 -> 7 -> 8
  Time: 8 units

Parallel Stream (4 cores):
  [1, 2, 3, 4, 5, 6, 7, 8]
  Thread 1: [1, 2]    Thread 2: [3, 4]    Thread 3: [5, 6]    Thread 4: [7, 8]
  Time: ~2 units (4x faster)
```

Creating Parallel Streams

```java
// From collection
List<Integer> list = List.of(1, 2, 3, 4, 5);
list.parallelStream()
    .filter(n -> n > 2)
    .collect(Collectors.toList());

// Convert sequential to parallel
list.stream()
    .parallel()
    .map(n -> n * 2)
    .collect(Collectors.toList());

// Check if stream is parallel
boolean isParallel = list.parallelStream().isParallel();  // true
```

When to Use Parallel Streams

Use Parallel When	Avoid Parallel When
Dataset > 10,000 elements	Dataset < 1,000 elements
CPU-intensive operations	I/O-bound operations (DB calls, API calls)
Independent operations (no shared state)	Operations with side effects
Simple, stateless transformations	Order-dependent processing
Splittable data sources (ArrayList, arrays)	Hard-to-split sources (LinkedList, Streams)

When NOT to Use Parallel Streams

```
// BAD: Shared mutable state
List<Integer> results = new ArrayList<>();  // NOT thread-safe!
numbers.parallelStream()
    .filter(n -> n > 0)
    .forEach(results::add);  // Race condition! Elements may be lost or
duplicated

// GOOD: Use collect instead
List<Integer> results = numbers.parallelStream()
    .filter(n -> n > 0)
    .collect(Collectors.toList());  // Thread-safe

// BAD: I/O-bound operations block ForkJoinPool
users.parallelStream()
    .forEach(user -> sendEmail(user));  // Blocks shared ForkJoinPool threads!

// BAD: Small dataset -- overhead > benefit
List.of(1, 2, 3).parallelStream()
    .map(n -> n * 2)   // Thread management overhead exceeds computation time
    .collect(Collectors.toList());
```

Custom ForkJoinPool for Parallel Streams

```
// Use custom pool to avoid blocking the shared ForkJoinPool
ForkJoinPool customPool = new ForkJoinPool(8);
try {
    List<Result> results = customPool.submit(() ->
        largeDataset.parallelStream()
            .map(this::heavyComputation)
            .collect(Collectors.toList())
```

```
    ).get();
} finally {
    customPool.shutdown();
}
```

10.2 Primitive Streams: IntStream, LongStream, DoubleStream

Why Primitive Streams Exist

```
Problem with Stream<Integer>:
  1. Each int is BOXED into Integer object (autoboxing)
  2. Each Integer is UNBOXED back to int for operations (autounboxing)
  3. Creates garbage objects that need GC
  4. Significant performance overhead for numeric operations

Solution: Primitive Streams
  IntStream, LongStream, DoubleStream
  - Work directly with primitive types
  - No boxing/unboxing
  - Provide specialized numeric methods (sum, average, etc.)
```

IntStream Methods

```
// Creating IntStream
IntStream range = IntStream.range(1, 10);          // 1 to 9 (exclusive end)
IntStream rangeClosed = IntStream.rangeClosed(1, 10); // 1 to 10 (inclusive
end)
IntStream fromArray = IntStream.of(1, 2, 3, 4, 5);
IntStream fromArray2 = Arrays.stream(new int[]{1, 2, 3});

// Specialized operations
int sum = IntStream.rangeClosed(1, 100).sum();                    // 5050
OptionalDouble avg = IntStream.of(10, 20, 30).average();          //
OptionalDouble[20.0]
OptionalInt max = IntStream.of(10, 20, 30).max();                 //
OptionalInt[30]
IntSummaryStatistics stats = IntStream.of(10, 20, 30).summaryStatistics();

// Converting between Stream<Integer> and IntStream
List<Integer> list = List.of(1, 2, 3);
IntStream intStream = list.stream().mapToInt(Integer::intValue);  // Stream ->
IntStream
```

```
Stream<Integer> boxed = intStream.boxed();                          // IntStream
-> Stream

// IntStream for character processing
String str = "Hello World";
IntStream chars = str.chars();                                      // IntStream
of char values
str.chars()
    .filter(Character::isUpperCase)
    .forEach(c -> System.out.print((char) c));  // "HW"
```

LongStream and DoubleStream

```
// LongStream -- for large numeric ranges
long sum = LongStream.rangeClosed(1, 1_000_000_000L).sum();

// DoubleStream -- for floating-point operations
double average = DoubleStream.of(1.5, 2.5, 3.5, 4.5).average().orElse(0.0);

// Converting mapToInt, mapToLong, mapToDouble
double totalSalary = employees.stream()
    .mapToDouble(Employee::getSalary)  // Stream<Employee> -> DoubleStream
    .sum();

// Statistics
DoubleSummaryStatistics stats = employees.stream()
    .mapToDouble(Employee::getSalary)
    .summaryStatistics();
```

Boxing vs Primitive Streams Performance

Operation	`Stream<Integer>`	`IntStream`
Storage	Boxed Integer objects on heap	Primitive int values
Memory	~16 bytes per element	4 bytes per element
Sum	`reduce(0, Integer::sum)`	`sum()` -- built-in
Average	Manual calculation needed	`average()` -- built-in
Performance	Slower (boxing/unboxing + GC)	Faster (no overhead)

10.3 Performance Considerations

Stream Performance Guidelines

```
1. AVOID UNNECESSARY BOXING
   BAD:  list.stream().map(n -> n * 2).reduce(0, Integer::sum)
   GOOD: list.stream().mapToInt(n -> n * 2).sum()

2. PLACE filter() BEFORE map()
   BAD:  stream.map(expensive).filter(simple)     // Maps ALL, then filters
   GOOD: stream.filter(simple).map(expensive)     // Filters first, maps fewer

3. USE SHORT-CIRCUIT OPERATIONS
   BAD:  stream.filter(predicate).count() > 0     // Processes ALL elements
   GOOD: stream.anyMatch(predicate)               // Stops at first match

4. AVOID sorted() ON LARGE STREAMS WITH limit()
   BAD:  stream.sorted().limit(10)                // Sorts ALL, takes 10
   Consider: Use a min-heap/PriorityQueue instead

5. PREFER collect() OVER forEach() + ADD
   BAD:  list = new ArrayList<>(); stream.forEach(list::add)
   GOOD: list = stream.collect(Collectors.toList())
```

Part 11: Real-World Use Cases

11.1 Filtering Large Datasets

```
// E-commerce: Filter products based on multiple criteria
public List<ProductDTO> searchProducts(SearchCriteria criteria) {
    return productRepository.findAll().stream()
        .filter(p -> criteria.getCategory() == null
            || p.getCategory().equals(criteria.getCategory()))
        .filter(p -> criteria.getMinPrice() == null
            || p.getPrice().compareTo(criteria.getMinPrice()) >= 0)
        .filter(p -> criteria.getMaxPrice() == null
            || p.getPrice().compareTo(criteria.getMaxPrice()) <= 0)
        .filter(p -> criteria.isInStock() == null || p.getStock() > 0)
        .sorted(Comparator.comparing(Product::getPrice))
        .skip((long) criteria.getPage() * criteria.getSize())
        .limit(criteria.getSize())
        .map(this::toDTO)
        .collect(Collectors.toList());
}
```

11.2 Transforming API Responses

```
// Microservice: Transform external API response to internal model
public List<InternalOrder> fetchAndTransformOrders(String customerId) {
    ExternalApiResponse response = externalApi.getOrders(customerId);

    return response.getData().stream()
        .filter(order -> order.getStatus() != null)
        .map(extOrder -> InternalOrder.builder()
            .orderId(extOrder.getId())
            .customerName(extOrder.getCustomer().getFullName())
            .totalAmount(extOrder.getItems().stream()
                .mapToDouble(item -> item.getPrice() * item.getQuantity())
                .sum())
            .itemCount(extOrder.getItems().size())
            .status(mapStatus(extOrder.getStatus()))
            .createdAt(Instant.parse(extOrder.getTimestamp()))
            .build())
```

```
        .sorted(Comparator.comparing(InternalOrder::getCreatedAt).reversed())
        .collect(Collectors.toList());
}
```

11.3 Grouping Database Results

```
// HR Analytics: Department-wise salary analysis
public Map<String, DepartmentStats> getDepartmentAnalytics() {
    List<Employee> employees = employeeRepository.findAll();

    return employees.stream()
        .collect(Collectors.groupingBy(
            Employee::getDepartment,
            Collectors.collectingAndThen(
                Collectors.toList(),
                emps -> new DepartmentStats(
                    emps.size(),

emps.stream().mapToDouble(Employee::getSalary).average().orElse(0),

emps.stream().mapToDouble(Employee::getSalary).max().orElse(0),

emps.stream().mapToDouble(Employee::getSalary).min().orElse(0),
                    emps.stream().mapToDouble(Employee::getSalary).sum()
                )
            )
        ));
}
```

11.4 Analytics Calculations

```
// Sales analytics: Monthly revenue trends
public Map<YearMonth, BigDecimal> getMonthlyRevenue(List<Order> orders) {
    return orders.stream()
        .filter(o -> o.getStatus() == OrderStatus.COMPLETED)
        .collect(Collectors.groupingBy(
            o -> YearMonth.from(o.getCompletedDate()),
            TreeMap::new,  // Sorted by month
            Collectors.reducing(
                BigDecimal.ZERO,
```

```
                Order::getAmount,
                BigDecimal::add
            )
        ));
}

// Top selling products
public List<ProductSalesDTO> getTopSellingProducts(int topN) {
    return orderItemRepository.findAll().stream()
        .collect(Collectors.groupingBy(
            OrderItem::getProductId,
            Collectors.summingLong(OrderItem::getQuantity)
        ))
        .entrySet().stream()
        .sorted(Map.Entry.<Long, Long>comparingByValue().reversed())
        .limit(topN)
        .map(entry -> new ProductSalesDTO(entry.getKey(), entry.getValue()))
        .collect(Collectors.toList());
}
```

11.5 Log Processing

```
// Process application logs and generate error summary
public Map<String, Long> analyzeErrors(String logFilePath) throws IOException {
    try (Stream<String> lines = Files.lines(Paths.get(logFilePath))) {
        return lines
            .filter(line -> line.contains("ERROR") ||
line.contains("EXCEPTION"))
            .map(line -> {
                // Extract exception class name
                int idx = line.indexOf("Exception");
                if (idx > 0) {
                    int start = line.lastIndexOf(' ', idx) + 1;
                    return line.substring(start, idx + "Exception".length());
                }
                return "UnknownError";
            })
            .collect(Collectors.groupingBy(
                Function.identity(),
                Collectors.counting()
            ))
            .entrySet().stream()
            .sorted(Map.Entry.<String, Long>comparingByValue().reversed())
            .collect(Collectors.toMap(
```

```
                Map.Entry::getKey,
                Map.Entry::getValue,
                (e1, e2) -> e1,
                LinkedHashMap::new  // Preserve sorted order
            ));
    }
}
```

Part 12: 50+ Coding Interview Problems Using Streams

Easy Problems

Problem 1: Find Duplicate Elements in a List

Problem: Given a list of integers, find all duplicate elements.

```
// Brute Force
public List<Integer> findDuplicatesBrute(List<Integer> list) {
    List<Integer> duplicates = new ArrayList<>();
    Set<Integer> seen = new HashSet<>();
    for (int num : list) {
        if (!seen.add(num)) {
            if (!duplicates.contains(num)) duplicates.add(num);
        }
    }
    return duplicates;
}

// Stream Solution
public List<Integer> findDuplicatesStream(List<Integer> list) {
    Set<Integer> seen = new HashSet<>();
    return list.stream()
        .filter(n -> !seen.add(n))  // add() returns false if already present
        .distinct()
        .collect(Collectors.toList());
}

// Alternative: Using groupingBy
public List<Integer> findDuplicatesGrouping(List<Integer> list) {
    return list.stream()
        .collect(Collectors.groupingBy(Function.identity(),
Collectors.counting()))
        .entrySet().stream()
        .filter(e -> e.getValue() > 1)
        .map(Map.Entry::getKey)
        .collect(Collectors.toList());
}
```

Problem 2: Find First Non-Repeated Character in a String

Problem: Given a string, find the first character that does not repeat.

```java
// Brute Force
public Character firstNonRepeatedBrute(String str) {
    Map<Character, Integer> countMap = new LinkedHashMap<>();
    for (char c : str.toCharArray()) {
        countMap.merge(c, 1, Integer::sum);
    }
    for (Map.Entry<Character, Integer> entry : countMap.entrySet()) {
        if (entry.getValue() == 1) return entry.getKey();
    }
    return null;
}

// Stream Solution
public Character firstNonRepeatedStream(String str) {
    return str.chars()
        .mapToObj(c -> (char) c)
        .collect(Collectors.groupingBy(Function.identity(),
            LinkedHashMap::new, Collectors.counting()))
        .entrySet().stream()
        .filter(e -> e.getValue() == 1)
        .map(Map.Entry::getKey)
        .findFirst()
        .orElse(null);
}
```

Problem 3: Remove Duplicates from a List

Problem: Remove duplicate elements from a list while preserving order.

```java
// Stream Solution
List<Integer> unique = list.stream()
    .distinct()
    .collect(Collectors.toList());
```

Problem 4: Find the Second Highest Number

Problem: Find the second largest number in a list.

```java
// Brute Force
public int secondHighestBrute(List<Integer> list) {
    int first = Integer.MIN_VALUE, second = Integer.MIN_VALUE;
    for (int n : list) {
        if (n > first) { second = first; first = n; }
        else if (n > second && n != first) { second = n; }
    }
    return second;
}

// Stream Solution
public Optional<Integer> secondHighestStream(List<Integer> list) {
    return list.stream()
        .distinct()
        .sorted(Comparator.reverseOrder())
        .skip(1)
        .findFirst();
}
```

Problem 5: Count Occurrences of Each Character

Problem: Given a string, count the frequency of each character.

```java
// Stream Solution
public Map<Character, Long> charFrequency(String str) {
    return str.chars()
        .mapToObj(c -> (char) c)
        .collect(Collectors.groupingBy(Function.identity(),
Collectors.counting()));
}
```

Problem 6: Find the Longest String in a List

```java
Optional<String> longest = strings.stream()
    .max(Comparator.comparingInt(String::length));
```

Problem 7: Convert List of Strings to Uppercase

```
List<String> upper = strings.stream()
    .map(String::toUpperCase)
    .collect(Collectors.toList());
```

Problem 8: Find Sum of All Even Numbers

```
int sum = numbers.stream()
    .filter(n -> n % 2 == 0)
    .mapToInt(Integer::intValue)
    .sum();
```

Problem 9: Check if All Elements are Positive

```
boolean allPositive = numbers.stream().allMatch(n -> n > 0);
```

Problem 10: Flatten a List of Lists

```
List<Integer> flat = listOfLists.stream()
    .flatMap(Collection::stream)
    .collect(Collectors.toList());
```

Medium Problems

Problem 11: Group Employees by Department

```
// Group and get names per department
Map<String, List<String>> namesByDept = employees.stream()
    .collect(Collectors.groupingBy(
        Employee::getDepartment,
        Collectors.mapping(Employee::getName, Collectors.toList())
    ));
```

Problem 12: Sort a Map by Value

```
Map<String, Integer> sortedByValue = unsortedMap.entrySet().stream()
    .sorted(Map.Entry.<String, Integer>comparingByValue().reversed())
    .collect(Collectors.toMap(
        Map.Entry::getKey,
        Map.Entry::getValue,
        (e1, e2) -> e1,
        LinkedHashMap::new
    ));
```

Problem 13: Find Average Salary by Department

```
Map<String, Double> avgSalary = employees.stream()
    .collect(Collectors.groupingBy(
        Employee::getDepartment,
        Collectors.averagingDouble(Employee::getSalary)
    ));
```

Problem 14: Partition Numbers into Even and Odd

```
Map<Boolean, List<Integer>> partitioned = numbers.stream()
    .collect(Collectors.partitioningBy(n -> n % 2 == 0));

List<Integer> evens = partitioned.get(true);
List<Integer> odds = partitioned.get(false);
```

Problem 15: Find Common Elements Between Two Lists

```
List<Integer> common = list1.stream()
    .filter(list2::contains)
    .distinct()
    .collect(Collectors.toList());

// More efficient with Set
Set<Integer> set2 = new HashSet<>(list2);
List<Integer> common = list1.stream()
```

```
    .filter(set2::contains)
    .collect(Collectors.toList());
```

Problem 16: Convert List to Map (id -> object)

```
Map<Long, Employee> employeeMap = employees.stream()
    .collect(Collectors.toMap(Employee::getId, Function.identity()));
```

Problem 17: Find the Most Frequent Element

```
Optional<Integer> mostFrequent = list.stream()
    .collect(Collectors.groupingBy(Function.identity(), Collectors.counting()))
    .entrySet().stream()
    .max(Map.Entry.comparingByValue())
    .map(Map.Entry::getKey);
```

Problem 18: Join List of Strings with Delimiter

```
String result = List.of("Java", "Streams", "API").stream()
    .collect(Collectors.joining(" - "));
// Output: "Java - Streams - API"
```

Problem 19: Find Employee with Highest Salary in Each Department

```
Map<String, Optional<Employee>> highestPaidByDept = employees.stream()
    .collect(Collectors.groupingBy(
        Employee::getDepartment,
        Collectors.maxBy(Comparator.comparingDouble(Employee::getSalary))
    ));
```

Problem 20: Merge Two Maps

```
Map<String, Integer> merged = Stream.of(map1, map2)
    .flatMap(map -> map.entrySet().stream())
    .collect(Collectors.toMap(
        Map.Entry::getKey,
        Map.Entry::getValue,
        Integer::sum  // Merge function: sum values for duplicate keys
    ));
```

Problem 21: Find All Palindromes in a List of Strings

```
List<String> palindromes = strings.stream()
    .filter(s -> s.equals(new StringBuilder(s).reverse().toString()))
    .collect(Collectors.toList());
```

Problem 22: Create a Frequency Map from a String Array

```
Map<String, Long> frequency = Arrays.stream(words)
    .collect(Collectors.groupingBy(
        String::toLowerCase,
        Collectors.counting()
    ));
```

Problem 23: Find Strings Starting with a Specific Letter

```
List<String> startsWithA = strings.stream()
    .filter(s -> s.startsWith("A"))
    .collect(Collectors.toList());
```

Problem 24: Find Numbers at Even Indices

```
List<Integer> evenIndexed = IntStream.range(0, list.size())
    .filter(i -> i % 2 == 0)
```

```
    .mapToObj(list::get)
    .collect(Collectors.toList());
```

Problem 25: Calculate Product of All Elements

```
int product = numbers.stream()
    .reduce(1, (a, b) -> a * b);
```

Problem 26: Find Nth Highest Salary

```
public Optional<Double> nthHighestSalary(List<Employee> employees, int n) {
    return employees.stream()
        .map(Employee::getSalary)
        .distinct()
        .sorted(Comparator.reverseOrder())
        .skip(n - 1)
        .findFirst();
}
```

Problem 27: Reverse Each Word in a Sentence

```
String reversed = Arrays.stream(sentence.split(" "))
    .map(word -> new StringBuilder(word).reverse().toString())
    .collect(Collectors.joining(" "));
```

Problem 28: Find Employees Who Joined in a Specific Year

```
List<Employee> joined2024 = employees.stream()
    .filter(e -> e.getJoiningDate().getYear() == 2024)
    .sorted(Comparator.comparing(Employee::getJoiningDate))
    .collect(Collectors.toList());
```

Problem 29: Sum Salary by Department

```
Map<String, Double> deptSalary = employees.stream()
    .collect(Collectors.groupingBy(
        Employee::getDepartment,
        Collectors.summingDouble(Employee::getSalary)
    ));
```

Problem 30: Find the Kth Smallest Element

```
public Optional<Integer> kthSmallest(List<Integer> list, int k) {
    return list.stream()
        .distinct()
        .sorted()
        .skip(k - 1)
        .findFirst();
}
```

Hard Problems

Problem 31: Find All Pairs That Sum to a Target

```
public List<int[]> findPairsWithSum(List<Integer> list, int target) {
    Set<Integer> seen = new HashSet<>();
    return list.stream()
        .filter(n -> {
            boolean hasPair = seen.contains(target - n);
            seen.add(n);
            return hasPair;
        })
        .map(n -> new int[]{target - n, n})
        .collect(Collectors.toList());
}
```

Problem 32: Flatten Deeply Nested List

```
// Recursive flattening using streams
public <T> Stream<T> flatten(List<?> list) {
```

```
    return list.stream()
        .flatMap(item -> item instanceof List
            ? flatten((List<?>) item)
            : Stream.of((T) item));
}

// Usage
List<Integer> flat = flatten(deeplyNested).collect(Collectors.toList());
```

Problem 33: Group Anagrams Together

```
Map<String, List<String>> anagrams = words.stream()
    .collect(Collectors.groupingBy(word -> {
        char[] chars = word.toLowerCase().toCharArray();
        Arrays.sort(chars);
        return new String(chars);
    }));
// Input:  ["eat", "tea", "tan", "ate", "nat", "bat"]
// Output: {aet=[eat, tea, ate], ant=[tan, nat], abt=[bat]}
```

Problem 34: Longest Consecutive Sequence

```
public int longestConsecutive(List<Integer> nums) {
    Set<Integer> set = new HashSet<>(nums);
    return set.stream()
        .filter(n -> !set.contains(n - 1))  // Start of a sequence
        .mapToInt(n -> {
            int count = 0;
            while (set.contains(n + count)) count++;
            return count;
        })
        .max()
        .orElse(0);
}
```

Problem 35: Top N Salaries Across All Departments

```
Map<String, List<Employee>> topNByDept = employees.stream()
    .collect(Collectors.groupingBy(Employee::getDepartment))
```

```
    .entrySet().stream()
    .collect(Collectors.toMap(
        Map.Entry::getKey,
        e -> e.getValue().stream()
            .sorted(Comparator.comparingDouble(Employee::getSalary).reversed())
            .limit(3)
            .collect(Collectors.toList())
    ));
```

Problem 36: Find the Intersection of Multiple Lists

```
public <T> List<T> intersection(List<List<T>> lists) {
    return lists.stream()
        .map(HashSet::new)
        .reduce((set1, set2) -> {
            set1.retainAll(set2);
            return set1;
        })
        .map(ArrayList::new)
        .orElse(new ArrayList<>());
}
```

Problem 37: Matrix Transposition Using Streams

```
int[][] matrix = {{1,2,3},{4,5,6},{7,8,9}};
int[][] transposed = IntStream.range(0, matrix[0].length)
    .mapToObj(col -> IntStream.range(0, matrix.length)
        .map(row -> matrix[row][col])
        .toArray())
    .toArray(int[][]::new);
```

Problem 38: Generate Fibonacci Series Using Streams

```
List<Long> fibonacci = Stream.iterate(new long[]{0, 1}, f -> new long[]{f[1],
f[0] + f[1]})
    .limit(20)
    .map(f -> f[0])
```

```
    .collect(Collectors.toList());
// Output: [0, 1, 1, 2, 3, 5, 8, 13, 21, 34, ...]
```

Problem 39: Word Count from File

```
public Map<String, Long> wordCount(String filePath) throws IOException {
    try (Stream<String> lines = Files.lines(Paths.get(filePath))) {
        return lines
            .flatMap(line -> Arrays.stream(line.split("\\W+")))
            .filter(word -> !word.isEmpty())
            .map(String::toLowerCase)
            .collect(Collectors.groupingBy(Function.identity(),
Collectors.counting()));
    }
}
```

Problem 40: Find Employees with Duplicate Names

```
List<String> duplicateNames = employees.stream()
    .collect(Collectors.groupingBy(Employee::getName, Collectors.counting()))
    .entrySet().stream()
    .filter(e -> e.getValue() > 1)
    .map(Map.Entry::getKey)
    .collect(Collectors.toList());
```

Problem 41: Custom Collector - Collect to ImmutableList

```
public static <T> Collector<T, ?, List<T>> toImmutableList() {
    return Collectors.collectingAndThen(
        Collectors.toList(),
        Collections::unmodifiableList
    );
}

List<String> immutable = names.stream()
    .filter(n -> n.length() > 3)
    .collect(toImmutableList());
```

Problem 42: Sliding Window Average

```
public List<Double> slidingWindowAverage(List<Integer> list, int windowSize) {
    return IntStream.rangeClosed(0, list.size() - windowSize)
        .mapToObj(i -> list.subList(i, i + windowSize).stream()
            .mapToInt(Integer::intValue)
            .average()
            .orElse(0.0))
        .collect(Collectors.toList());
}
```

Problem 43: Find All Substrings of Length N

```
List<String> substrings = IntStream.rangeClosed(0, str.length() - n)
    .mapToObj(i -> str.substring(i, i + n))
    .distinct()
    .collect(Collectors.toList());
```

Problem 44: Zip Two Lists Together

```
public <A, B> List<Map.Entry<A, B>> zip(List<A> list1, List<B> list2) {
    return IntStream.range(0, Math.min(list1.size(), list2.size()))
        .mapToObj(i -> Map.entry(list1.get(i), list2.get(i)))
        .collect(Collectors.toList());
}
```

Problem 45: Running Total (Cumulative Sum)

```
List<Integer> numbers = List.of(1, 2, 3, 4, 5);
List<Integer> runningTotal = new ArrayList<>();
numbers.stream().reduce(0, (sum, n) -> {
    int newSum = sum + n;
    runningTotal.add(newSum);
    return newSum;
```

```
});
// Output: [1, 3, 6, 10, 15]
```

Problem 46: Find the Longest Common Prefix

```
public String longestCommonPrefix(List<String> strings) {
    return strings.stream()
        .reduce((s1, s2) -> {
            int minLen = Math.min(s1.length(), s2.length());
            int i = 0;
            while (i < minLen && s1.charAt(i) == s2.charAt(i)) i++;
            return s1.substring(0, i);
        })
        .orElse("");
}
```

Problem 47: Convert Nested Map to Flat Map

```
Map<String, Map<String, Integer>> nested = // ...
Map<String, Integer> flat = nested.entrySet().stream()
    .flatMap(outer -> outer.getValue().entrySet().stream()
        .map(inner -> Map.entry(
            outer.getKey() + "." + inner.getKey(),
            inner.getValue()
        )))
    .collect(Collectors.toMap(Map.Entry::getKey, Map.Entry::getValue));
```

Problem 48: Prime Number Check and Filtering

```
Predicate<Integer> isPrime = n -> n > 1 &&
    IntStream.rangeClosed(2, (int) Math.sqrt(n))
        .noneMatch(i -> n % i == 0);

List<Integer> primes = IntStream.rangeClosed(2, 100)
    .filter(n -> isPrime.test(n))
    .boxed()
    .collect(Collectors.toList());
```

Problem 49: Cartesian Product of Two Lists

```
List<String> cartesian = list1.stream()
    .flatMap(a -> list2.stream().map(b -> a + "-" + b))
    .collect(Collectors.toList());
// [1,2] x [A,B] -> [1-A, 1-B, 2-A, 2-B]
```

Problem 50: Complex Employee Report

```
// Generate a report: Department -> {average salary, employee count, names}
Map<String, Map<String, Object>> report = employees.stream()
    .collect(Collectors.groupingBy(Employee::getDepartment))
    .entrySet().stream()
    .collect(Collectors.toMap(
        Map.Entry::getKey,
        e -> {
            List<Employee> deptEmps = e.getValue();
            Map<String, Object> stats = new LinkedHashMap<>();
            stats.put("count", deptEmps.size());
            stats.put("avgSalary", deptEmps.stream()
                .mapToDouble(Employee::getSalary).average().orElse(0));
            stats.put("maxSalary", deptEmps.stream()
                .mapToDouble(Employee::getSalary).max().orElse(0));
            stats.put("employees", deptEmps.stream()
                .map(Employee::getName).collect(Collectors.joining(", ")));
            return stats;
        }
    ));
```

Problem 51: Find Missing Numbers in a Range

```
Set<Integer> existing = new HashSet<>(list);
List<Integer> missing = IntStream.rangeClosed(1, 100)
    .filter(n -> !existing.contains(n))
    .boxed()
    .collect(Collectors.toList());
```

Problem 52: Character with Maximum Occurrences

```
Optional<Map.Entry<Character, Long>> maxChar = str.chars()
    .mapToObj(c -> (char) c)
    .collect(Collectors.groupingBy(Function.identity(), Collectors.counting()))
    .entrySet().stream()
    .max(Map.Entry.comparingByValue());
```

Part 13: Stream Best Practices

13.1 When NOT to Use Streams

```
DO NOT use streams when:
  1. Simple operations -- a for loop is clearer for simple iterations
  2. Performance-critical code -- streams add overhead for small datasets
  3. You need index-based access -- streams don't support indexing
  4. You need to modify the source -- streams are for read-only processing
  5. Complex stateful operations -- state management in streams leads to bugs
  6. Exception handling is required -- checked exceptions in lambdas are messy
```

Example: When a For Loop is Better

```java
// Stream -- overcomplicated for this simple case
boolean found = list.stream().anyMatch(item -> item.equals(target));

// Simple contains -- clearer
boolean found = list.contains(target);

// Stream -- poor for indexed operations
IntStream.range(0, list.size())
    .forEach(i -> list.set(i, list.get(i) * 2));

// For loop -- clearer and more natural
for (int i = 0; i < list.size(); i++) {
    list.set(i, list.get(i) * 2);
}
```

13.2 Readability Considerations

```java
// BAD: Dense, unreadable stream
var r = emps.stream().filter(e->e.getS()>50000&&e.getD().equals("IT")).map(e-
>e.getN()).sorted().limit(5).collect(Collectors.toList());

// GOOD: Well-formatted, readable stream
```

```
List<String> topITEmployees = employees.stream()
    .filter(emp -> emp.getSalary() > 50000)
    .filter(emp -> "IT".equals(emp.getDepartment()))
    .map(Employee::getName)
    .sorted()
    .limit(5)
    .collect(Collectors.toList());

// GOOD: Extract complex predicates
Predicate<Employee> isHighPaidIT = emp ->
    emp.getSalary() > 50000 && "IT".equals(emp.getDepartment());

List<String> topITEmployees = employees.stream()
    .filter(isHighPaidIT)
    .map(Employee::getName)
    .sorted()
    .limit(5)
    .collect(Collectors.toList());
```

13.3 Performance Pitfalls

Pitfall	Problem	Solution
Unnecessary boxing	`Stream<Integer>` instead of `IntStream`	Use `mapToInt()`, `IntStream.range()`
Sorting before filtering	Sorts entire dataset unnecessarily	Place `filter()` before `sorted()`
Using `stream().count()` for size	Creates a stream just to count	Use `collection.size()`
`forEach` for accumulation	Not thread-safe, not functional	Use `collect()`
Not closing resource streams	Memory/resource leak with `Files.lines()`	Always use try-with-resources
Overusing parallel streams	Thread overhead > computation benefit	Only for large CPU-bound work

13.4 Debugging Streams

```
// Technique 1: Use peek() for logging
List<String> result = employees.stream()
```

```
    .peek(e -> log.debug("Before filter: {}", e))
    .filter(e -> e.getSalary() > 50000)
    .peek(e -> log.debug("After filter: {}", e))
    .map(Employee::getName)
    .peek(n -> log.debug("After map: {}", n))
    .collect(Collectors.toList());

// Technique 2: Break pipeline into steps
Stream<Employee> filtered = employees.stream()
    .filter(e -> e.getSalary() > 50000);

Stream<String> mapped = filtered.map(Employee::getName);

List<String> result = mapped.collect(Collectors.toList());

// Technique 3: Use IDE debugger (IntelliJ's Stream Debugger)
// IntelliJ IDEA: Run -> Debug, then click "Trace Current Stream Chain"
```

Part 14: Common Interview Questions

Q1: What is lazy evaluation in Java Streams?

Answer: Lazy evaluation means intermediate operations (like `filter()`, `map()`, `sorted()`) are NOT executed immediately when called. They are only executed when a terminal operation (like `collect()`, `forEach()`, `count()`) is invoked. This allows the stream pipeline to optimize execution -- for example, if `findFirst()` is the terminal operation, the stream stops processing after finding the first matching element rather than processing the entire collection.

Q2: Difference between `map()` and `flatMap()`?

Answer:

- `map()`: One-to-one transformation. Each element maps to exactly one result. `Stream<T>` -> `Stream<R>`.
- `flatMap()`: One-to-many transformation + flattening. Each element maps to a stream of zero or more results, and all resulting streams are merged into one. `Stream<T>` -> `Stream<R>` (flattened).

Example: If you have a list of sentences and want individual words, `map(s -> s.split(" "))` gives `Stream<String[]>`, while `flatMap(s -> Arrays.stream(s.split(" ")))` gives `Stream<String>`.

Q3: Difference between `findFirst()` and `findAny()`?

Answer:

- `findFirst()`: Always returns the first element in encounter order. In parallel streams, it constrains processing to maintain order, reducing parallelism.
- `findAny()`: Returns any element, whichever is found first. In parallel streams, it can return whichever element any thread finds first, providing better performance.
- In sequential streams, both behave identically.

Q4: Difference between `reduce()` and `collect()`?

Answer:

- `reduce()` : Immutable reduction. Combines elements by replacing the accumulator (good for sum, product, concatenation).
- `collect()` : Mutable reduction. Accumulates into a mutable container like `ArrayList` , `StringBuilder` , `HashMap` .
- Use `reduce()` for computing single values. Use `collect()` for building collections or complex results.

Q5: Difference between `stream()` and `parallelStream()` ?

Answer:

- `stream()` : Sequential processing. Elements are processed one at a time in a single thread.
- `parallelStream()` : Parallel processing. Data is split into chunks and processed concurrently using the ForkJoinPool.
- `parallelStream()` is NOT always faster -- it adds thread management overhead. Use it only for large datasets (>10K elements) with CPU-intensive operations.

Q6: Can a stream be reused?

Answer: No. Once a terminal operation is called on a stream, it is consumed and cannot be reused. Attempting to reuse it throws `IllegalStateException` . To process the same data again, create a new stream from the source.

Q7: What is the difference between intermediate and terminal operations?

Answer:

- **Intermediate operations**: Return a new `Stream` , are lazy (not executed until terminal), and can be chained. Examples: `filter()` , `map()` , `sorted()` .
- **Terminal operations**: Trigger pipeline execution, produce a result or side effect, and consume the stream. Examples: `collect()` , `forEach()` , `reduce()` .

Q8: What are stateful vs stateless operations?

Answer:

- **Stateless**: Process each element independently -- `filter()`, `map()`, `flatMap()`, `peek()`.
- **Stateful**: Need to see all elements or maintain state -- `sorted()`, `distinct()`, `limit()`, `skip()`.
- Stateful operations can be less efficient in parallel streams because they may require synchronization.

Q9: How does `Optional` relate to streams?

Answer: Several terminal operations return `Optional` to safely handle potentially empty results: `findFirst()`, `findAny()`, `min()`, `max()`, `reduce()` (without identity). `Optional` forces the caller to handle the absence of a value, preventing `NullPointerException`.

Q10: What is short-circuit evaluation in streams?

Answer: Some operations don't need to process all elements:

- `findFirst()`, `findAny()`: Stop after finding one match
- `anyMatch()`: Stops at first `true`
- `allMatch()`: Stops at first `false`
- `noneMatch()`: Stops at first `true`
- `limit(n)`: Stops after n elements

This optimization is especially valuable on large or infinite streams.

Q11: How to handle checked exceptions in streams?

Answer: Lambdas in streams don't support checked exceptions directly. Solutions:

1. Wrap in try-catch inside the lambda
2. Create a wrapper method that converts checked to unchecked exceptions
3. Use a utility function

```java
// Wrapper approach
private <T, R> Function<T, R> wrap(ThrowingFunction<T, R> fn) {
    return t -> {
        try { return fn.apply(t); }
        catch (Exception e) { throw new RuntimeException(e); }
    };
}

list.stream().map(wrap(this::methodThatThrows)).collect(Collectors.toList());
```

Q12: What is the difference between `Collection.stream()` and `Stream.of()`?

Answer:

- `Collection.stream()`: Creates a stream from an existing collection (List, Set, etc.)
- `Stream.of()`: Creates a stream from explicitly provided elements or an array
- Both create sequential streams; the source differs.

Part 15: Practice Section

30 Additional Coding Problems (Without Solutions)

Solve these problems using Java Streams to test your mastery.

Easy

1. Given a list of integers, find the maximum value.
2. Given a list of strings, filter those that contain a specific substring.
3. Given a list of integers, find the count of numbers greater than a given threshold.
4. Given a list of employee objects, find the names of employees in a specific department.
5. Convert a list of integers to a comma-separated string.
6. Given a list of strings, find the shortest string.
7. Given two lists of integers, find elements present in the first but not in the second.
8. Square each element in a list and collect to a new list.
9. Given a list of strings, remove all null and empty values.
10. Find the sum of digits of a number using streams.

Medium

11. Given a list of transactions, find total transaction amount per customer.
12. Find the top 3 most expensive products from a product list.
13. Given a list of sentences, find the total word count.
14. Generate a list of the first 50 prime numbers using streams.
15. Given a map of student names to grades, find students with grades above 90.
16. Given a list of orders, group by month and calculate total revenue per month.
17. Implement a case-insensitive distinct operation on a list of strings.
18. Given a list of employees, find the department with the highest average salary.
19. Create a stream pipeline that reads a CSV file, parses it, filters by a column value, and writes matching rows to a new file.
20. Given a list of strings, find the most common starting letter.

Hard

21. Implement a custom `Collector` that collects elements into an unmodifiable `Map`.
22. Given a list of integers, find all sub-arrays of length K that have an average greater than a threshold.
23. Given a tree structure represented as nested objects, flatten it into a list using streams.
24. Implement a parallel stream-based solution to count word frequencies in multiple files simultaneously.

25. Create a stream-based solution for the "Top K Frequent Elements" problem.
26. Implement a lazy paginator that fetches and processes records in chunks using streams.
27. Given a list of JSON strings, parse them, validate, transform, and group by a field -- all using streams.
28. Create a custom spliterator for iterating a binary tree in-order as a stream.
29. Implement a stream-based solution for detecting cyclic dependencies in a directed graph.
30. Build a data pipeline that reads from multiple sources (files, database, API), merges, deduplicates, transforms, and writes to a unified output -- using streams.

Appendix: Quick Reference Card

Stream Pipeline

```
Source -> [filter -> map -> sorted -> ...] -> Terminal Operation -> Result
         \___ Intermediate (LAZY) ___/     \____ EAGER ____/
```

Most Used Patterns

```
// Filter + Map + Collect
list.stream().filter(predicate).map(function).collect(Collectors.toList());

// Group By
list.stream().collect(Collectors.groupingBy(classifier));

// Sort + Limit (Top N)
list.stream().sorted(comparator.reversed()).limit(n).collect(Collectors.toList())

// Reduce (Aggregate)
list.stream().mapToInt(mapper).sum();

// Any/All/None Match
list.stream().anyMatch(predicate);

// Find First
list.stream().filter(predicate).findFirst().orElse(defaultValue);
```

Key Rules

Rule	Explanation
Streams are lazy	Intermediate ops execute only when terminal op is called
Streams are consumable	A stream can be used only ONCE
Don't modify source	Stream operations should not modify the source collection

filter before map	Reduces elements to transform, improving performance
Use primitive streams	`IntStream`, `LongStream` avoid boxing overhead
Close resource streams	Use try-with-resources for `Files.lines()`
Avoid side effects	Keep stream operations pure and functional

---

End of Guide -- Mastering Java Streams and Functional Programming